

The editor is a view: a Figma-feel canvas, tree, inspector and library over the one write path, with no new way to write

The win: you can explain, to an interviewer or to yourself, what a React component is and why the whole editor re-renders from one replica instead of mutating widgets; what `useSyncExternalStore` is for and why the peer is opened in an effect and never during render; why the selection is derived and not stored; how a CSS custom property differs from a Sass variable and why the dark theme is one attribute on `<html>` and not a media query; what "screen = world × zoom + offset" buys and why one `transform` on one layer beats re-laying anything out; how the deepest box wins a hit-test; why a drop on the layers tree is exactly one `move_node` with the index counted after the node is taken out, and what a remove-then-add would have broken; why the tree refuses only the structurally impossible and never predicts `validate()`; how the inspector is the manifest rendered, one control per prop *kind*, and why a typed value is one op on blur, never one per keystroke; how the library turns three intents into one `add_node` with an id minted once; why cursors ride the wire in world coordinates and how the AI, which has no pointer, gets a cursor from the ops the room ordered; how one shortcut registry drives both the handlers and the `?` sheet; how a native `<dialog>` traps focus for free; why the members list is *not* optimistic while the document is; what a role in the session is (a hint for the screen, never authority); how axe, screenshots and a routed WebSocket that counts frames guard all of it; where this sits against Swing, JavaFX, Qt and Figma; and you can point at the line of code for every claim.

Verification status, honestly. Every commit in this chapter passed lint, the type checker and the unit tests (theme.test with the WCAG contrast of every token pair in both themes, viewport.test, selection.test, designSystem.test, layer-moves.test, controls.test over the committed manifest, spaces.test, library-adds.test, cursors.test, shortcuts.test with the no-collision rule, preview-split.test, members.test, links.test, the contracts, the peer's onOp, the route-role table). The end-to-end specs (shell-light-dark, canvas-zoom-pan-select, layers-drag-and-keyboard-reorder, inspector-edits-propagate, library-drag-to-canvas, cursors-agree-across-zoom, ai-cursor-on-touched-node, preview-split-device-frames, shortcut-sheet, members-role-change, share-dialog) and the one integration suite (members-shares.int) are written and committed and *have not been run*: the host was too loaded to run Docker suites beside other work. The screenshot baselines (shell-light.png, shell-dark.png, home-dark.png, home.png) change with every bead of this epic and must be regenerated in that pass. E10.6 was also seen live with two browsers at 106 % and 32 %, and E10.8 on a source api and sync node; those are eyes, not a gate. Where this page says "measured", it was measured by a unit test that ran; where it says "the spec asserts", the spec exists and is unrun. Section 12 is the by-hand demo to run first.

How to use this page. Every code block is cut from the repo at commit f21d6d2, so what you read is what is committed. Section 1 is React from zero and the thesis of the chapter; sections 3, 4 and 7 are the ones a "build a design tool" interview digs into; section 5 is the one a "render a form from a schema" question turns on; section 9 is the one a security review asks about (what the screen may decide); section 10 is the comparison you will be asked to make.

```
pnpm exec vitest run apps/web/src # every pure half of the editor,
in seconds
pnpm exec vite --config apps/web/vite.config.ts --port 5199 # the editor on a port of your own,
beside the demo's 5173
pnpm exec playwright test e2e/layers-drag-and-keyboard-reorder.spec.ts # counts the move_node frames on
a real WebSocket
```

Try it, once the stack is up: section 12 walks the theme, the zoom, a drag with the DevTools network tab open, the inspector, the library, two browsers' cursors, the sheet and Share, then the suites, with the exact commands and what each should show.

The map: eight beads, one write path, and where each gesture becomes one op

- 1 · React from zero: a component is a function of its inputs, and the editor is a function of the replica
- 2 · Tokens, theming and contrast as a test
- 3 · Coordinate systems: one transform, outlines in world space, and the deepest box wins
- 4 · The layers tree: an ARIA tree whose selection is the canvas's, and a drop that is one move_node
- 5 · The inspector is the manifest, rendered: one control per prop kind, one op per edit
- 6 · The library: three intents, one Slot, one add_node, an id minted once
- 7 · Presence in world coordinates, and a cursor for a peer that has no pointer
- 8 · One shortcut registry, a native dialog, and a preview that is the device's real width
- 9 · Screens for what the api already does: a role as a hint, and a list that is never guessed
- 10 · Trade-offs and comparisons: where this editor sits
- 11 · Testing a screen: pure halves as arithmetic, a WebSocket that counts frames, axe in both themes
- 12 · By hand: the theme, the zoom, a drag with the network tab open, two cursors, the sheet, Share
- 13 · What the type system could not see, epic 10 edition

Your turn: four drills

Recall, then test yourself

The map: eight beads, one write path, and where each gesture becomes one op

Until this epic the frontend only drove the backend: a toolbar of "Add X" buttons, a list of "Select X" buttons, a form per node. Epic 10 makes it an editor that feels like Figma (owner decision, 2026-09-30) and the whole point of the chapter is what did *not* change while it did: the four ops (keystone 3), the one write path through `packages/peer-client` (keystone 2), the replica's two documents, the room's verdict. Every pane you will read about is a *view* over the replica, and every gesture, however much geometry it takes to read, ends as exactly one call to `peer.submit(op)`.

STEP	PURE MODULE	COMPONENT	WHAT IT DOES
1	apps/web/src/tokens.css	apps/web/src/useTheme.ts	every colour, size and space as a custom property; the dark block keyed on <code>html[data-theme]</code> , set before the first paint; <code>theme.test</code> recomputes WCAG AA for every text/surface pair from the CSS itself; <code>Shell.tsx</code> owns only where things go
2	apps/web/src/viewport.ts	apps/web/src/Surface.tsx	the sample app's REAL components inside an <code>inert</code> frame on an infinite sheet; <code>screen = world × zoom + offset</code> as one <code>transform</code> on one layer; outlines measured after render into world coordinates; the deepest box wins the hit-test (<code>selection.ts</code>)
3	apps/web/src/layer-moves.ts	apps/web/src/LayersPanel.tsx	an ARIA tree whose selection is the canvas's, both ways; a drop or Alt+arrow becomes ONE <code>move_node</code> whose index is counted after the node is taken out; refuses only the structurally impossible, never predicts <code>validate()</code>
4	apps/web/src/controls.ts	apps/web/src/Inspector.tsx	the manifest rendered: one control per prop KIND, layout props by name and kind, defaults as placeholders, Reset as <code>set_prop null</code> ; a typed value is one op on blur or Enter; a refusal repeated beside its control by <code>aria-describedby</code> ; <code>spaces.ts</code> shades gap and padding on the canvas
5	apps/web/src/library-adds.ts	apps/web/src/LibraryPanel.tsx	a listbox of tiles, each a small real render; a drag onto a row or the canvas, Enter or a click all end in a Slot and one <code>add_node</code> ; the id minted once in <code>Canvas.tsx</code> ; a refused add sends the selection back to the page
6	apps/web/src/cursors.ts	packages/peer-client/src/peer.ts	cursors ride the wire as world points bounded at $\pm 1e6$ and are drawn inside <code>.world</code> ; the AI has no pointer, so <code>onOp</code> (told after each ordered op is applied) anchors its mark to the node its last op touched; a still cursor fades by the clock
7	apps/web/src/shortcuts.ts	apps/web/src/Preview.tsx	ONE registry drives every keydown handler and the <code>?</code> sheet; a test refuses two rows one keydown could mean; the preview in a device frame of the device's real CSS width scaled by one transform; a native <code><dialog></code> for the sheet
8	apps/web/src/OrigView.tsx	apps/web/src/ShareDialog.tsx	two read routes in the E8.2 role table; the session carries a <code>role</code> beside the token (a hint for the screen, never authority); the members list and the shares are NOT optimistic: every row is the api's answer, and a 403 re-reads the world

Four ideas carry the epic. **The UI is a function of the replica** (section 1): no pane holds a copy of the document; React re-renders every pane from `peer.doc` when the replica's `revision` moves, which is why another person's move appears in the tree, on the canvas and in the inspector at once, and why the selection is derived (a node someone removed is simply the page again). **Geometry in, one op out** (sections 3 to 6): a pointer's height on a row, a point on an infinite sheet, a blur on a number field are read by pure modules into a Drop, a Slot or a value, and each becomes one of the four ops through the one `submit`; nothing in the editor knows a second way to write. **Refuse only the impossible** (section 4): the tree, the library and the inspector build every op the document's shape allows and let the replica and

the room say no, so `validate()` has one copy. **The screen decides nothing** (section 9): a role in the session shows or hides a button; the api and the room read the row for themselves on every request and every op.

What this chapter assumes: lesson 2's peer-client (the replica's `confirmed` and `optimistic`, `submit`, the room's verdict as `onRejected`) and lesson 8's roles. What it teaches from zero: React itself, CSS custom properties, coordinate transforms, the WAI-ARIA tree, listbox and dialog patterns. If you have written a Swing, JavaFX or Qt application, section 1 and section 10 are where your instincts will be wrong in useful ways.

1 · React from zero: a component is a function of its inputs, and the editor is a function of the replica

You know the desktop model: a widget is an object with state, an event mutates the object (`label.setText(...)`, `tree.getModel().insertNodeInto(...)`), and the toolkit repaints what changed. React inverts it. A *component* is a function from props (its inputs) to a description of what should be on the screen (JSX, which compiles to calls that build a tree of plain objects); when an input changes, React calls the function again and *diffs* the new description against the last one, touching only the DOM nodes that differ. You never say "update this label"; you say "given this document, here is the whole inspector", and React works out that one `<input value>` changed. State that belongs to a component (a draft in a text field, whether a row is collapsed) lives in a *hook*, `useState`, whose setter schedules a re-render; state that lives *outside* React (a WebSocket, the OS theme, a clock) gets in through `useSyncExternalStore`, which is the whole trick of this app.

FROM `apps/web/src/usePeer.ts`

```
function openStore(documentId: string, onRejected: (rejection: Rejection) => void, onOp: (message: SequencedOp) => void,
onSession: (session: SessionResponse) => void) {
  const listeners = new Set<() => void>();
  const tell = (): void => { for (const listener of listeners) listener(); };
  // Every session minted passes here (a reconnect too), so the role the editor shows controls by is the api's latest word.
  const session = async (): Promise<SessionResponse | null> => {
    const minted = await openSession(documentId);
    if (minted) onSession(minted);
    return minted;
  };
  const peer = connectPeer({ manifest, session, onChange: tell, onStatus: tell, onRejected, onOp });
  return {
    peer,
    subscribe: (listener: () => void) => { listeners.add(listener); return () => { listeners.delete(listener); }; },
    // peer.doc is edited IN PLACE and never changes identity, so the snapshot is built from what does move.
    snapshot: () =>
    ` ${String(peer.revision)}: ${String(peer.pendingCount)}: ${peer.status}: ${String(peer.readOnly)}: ${String(peer.presenceRevision)}
  `,
  };
}
```

Read `openStore` as an adapter. The peer changes on its own (a message arrives, a reconnect happens); React cannot see that.

`useSyncExternalStore(subscribe, getSnapshot)` asks for two things: how to be told something changed (`subscribe` registers a listener; the peer's `onChange` and `onStatus` call every listener) and how to read a value that React compares with `Object.is` to decide whether to re-render (`snapshot`). The document is edited *in place* by the replica, so `peer.doc` is the same object before and after an op and would never look different; the snapshot is therefore a string of the numbers that do move: `revision`, `pendingCount`, `status`, `readOnly`, `presenceRevision`. A new string, a re-render of `Canvas`; the same string, nothing.

FROM `packages/peer-client/src/replica.ts`

```
* It holds TWO documents:
*   confirmed   what the server has said, in the server's order. Never guessed at.
*   optimistic confirmed + every op of ours the server has not answered yet. What the user sees.
* The invariant (tested as a property): optimistic == confirmed, then each pending op, in order.
*/
```

That is why the editor can be a view: the replica already holds what the person should see (`optimistic`: what the server said plus every op of ours it has not answered yet, an invariant tested as a property in lesson 2), and `revision` is bumped whenever it changes. Every pane renders from `peer.doc`; none keeps a copy; so a move made in the tree is on the canvas in the same render, and a move made by someone else, arriving as a message, is in the tree, on the canvas and in the inspector in the render the message causes.

FROM apps/web/src/usePeer.ts

```
export function usePeer(documentId: string, onOp?: (message: SequencedOp) => void) {
  const [refusals, setRefusals] = useState<Refusal[]>([]);
  const [store, setStore] = useState<ReturnType<typeof openStore>>();
  const [role, setRole] = useState<Role>();
  const refuse = (reason: Reason, op?: Op): void => { setRefusals((before) => [...before, { id: crypto.randomUUID(), reason,
  ...(op === undefined ? {} : { op }) }]); };
  const latestOnOp = useRef(onOp);
  latestOnOp.current = onOp;
  useEffect(() => {
    // 'quiet' (someone else removed the node first) is not news: the canvas already shows it (F5).
    const opened = openStore(documentId, (rejection) => { if (!rejection.quiet) refuse(rejection.reason, rejection.op); },
    (message) => latestOnOp.current?.(message), (session) => { setRole(session.role); });
    setStore(opened);
    return () => { opened.peer.close(); };
  }, [documentId]);
  useSyncExternalStore((store ?? NOTHING).subscribe, (store ?? NOTHING).snapshot);
  return { peer: store?.peer, role, refusals, refuse, dismiss: (id: string): void => { setRefusals((before) =>
  before.filter((each) => each.id !== id)); } };
}
```

Two rules of the hook are load-bearing. **Side effects go in `useEffect`, never in the render body.** A render is a pure computation React may run and throw away (StrictMode does this on purpose in development, twice, to catch exactly this), so opening a WebSocket during render would open two; an effect runs after the commit, once per change of its dependencies (`[documentId]`), and returns a cleanup that closes the peer when the document changes or the component unmounts. **A callback that may change every render is held in a `useRef`** (`latestOnOp`): a ref is a box React does not watch, so `Canvas` may pass a fresh arrow function each render without the effect re-running and the peer being reopened. The Java analogue is a listener registered once in `addNotify` and removed in `removeNotify`; the Python analogue is a signal connected in `__init__` and disconnected in `closeEvent`; the difference is that React will call your render function far more often than a toolkit calls `paint`, and it must be safe to.

FROM apps/web/src/Canvas.tsx

```
// DERIVED, not stored: if someone else removes the selected node, the selection is simply the page again.
const node = doc.nodes[wanted] ?? root;
```

Derive, do not store. The canvas keeps only *which id the person wanted* (`wanted`); the node shown is `doc.nodes[wanted] ?? root`, computed every render. When someone else removes the selected node, no event handler has to notice: the next render finds no such node and the selection is the page, and one live sentence says so. Every derived value in this file (the labels, which nodes are containers, who has what selected, the AI's anchor) is recomputed from the document on each render, which sounds wasteful to a Swing programmer and is the reason the panes can never disagree: there is no second copy to fall out of date. The `ponytail` in `Surface.tsx` says where this stops paying: every node re-rendered on every change is fine at 64 levels and small documents; `memo` per node keyed on the node object is the upgrade.

FROM apps/web/src/Canvas.tsx

```
const submit = (op: Op): boolean => {
  if (op.type === "set_prop") for (const each of refusals) if (each.op?.type === "set_prop" && each.op.nodeId === op.nodeId
  && each.op.key === op.key) dismiss(each.id);
  const result = peer.submit(op);
  if (!result.ok) { refusedFrom.current = document.activeElement instanceof HTMLElement ? document.activeElement : null;
  refuse(result.reason, op); }
  return result.ok;
};
```

And here is the one write path as the editor sees it. Every pane receives this `submit` as a prop; the replica's verdict comes back synchronously (`result.ok` `false`: a notice at once, and for an add, the selection stays on the page); the room's verdict comes back later through `onRejected`, which `usePeer` turns into the same kind of notice. There is no other function in `apps/web` that can change the document.

FROM apps/web/src/Surface.tsx

```
// A ref, not state: the drag's last point changes with every pointer event and nothing renders from it.
const drag = useRef<Drag | null>(null);
```

Last, **state versus ref**, the distinction a React beginner gets wrong most. `useState` is for values the screen renders from: change it and React re-renders. `useRef` is for values that change without anything on the screen depending on them: the last pointer position during a pan, a press that may or may not become a drag, "focus the tab stop once it has rendered". A drag's origin put in state would re-render the whole canvas sixty times a second for nothing; in a ref it costs a property write.

TYPESCRIPT + REACT (THIS REPO)	JAVA (SWING, JAVAFX)	PYTHON (QT, TK)
<code>function Inspector(props) { return <aside>...</aside>; }</code> : a function from inputs to a description	<code>class Inspector extends JPanel</code> with fields and setters	<code>class Inspector(QWidget)</code> with attributes and slots
<code>useState</code> : change it and the function runs again	<code>setText</code> then <code>repaint()</code>	<code>setText</code> ; Qt repaints
<code>useSyncExternalStore(subscribe, snapshot)</code>	a <code>PropertyChangeListener</code> that calls <code>SwingUtilities.invokeLater(repaint)</code>	a signal connected to <code>update()</code>
<code>useEffect(() => { open(); return close; }, [documentId])</code>	<code>addNotify</code> / <code>removeNotify</code>	<code>showEvent</code> / <code>closeEvent</code>
<code>useRef</code> : a box React does not watch	a plain field	a plain attribute
re-render everything, diff the DOM	repaint the damaged region	repaint the damaged region

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

StrictMode rendered twice and would have opened two peers.

An early version opened the peer in `useMemo` "so it is created once". In development React runs the render twice and keeps one result, so two WebSockets opened and one leaked. Opening in `useEffect` with a cleanup fixed it and is the rule now (lesson 2's e2-5a memory). The instinct from Java ("construct it in the constructor") is exactly wrong here: a render is not a constructor, it is a `paint`.

2 · Tokens, theming and contrast as a test

A CSS *custom property* (`--accent: #1d4ed8`) is a variable that lives in the cascade: it is set on an element, inherited by its descendants, and read anywhere below with `var(--accent)` . Unlike a Sass variable, which is substituted at build time, it is resolved at *style time*, per element, so changing it on `<html>` recolours the page without touching a rule, and setting it again on a subtree recolours only that subtree. `tokens.css` declares every colour, size, space and radius the app uses as a custom property and nothing else declares one; `app.css` only refers to them.

FROM `apps/web/src/tokens.css`

```
/* the ONE accent */
--accent: #1d4ed8;
--on-accent: #ffffff;
--accent-soft: #e8e8ff;
--focus: var(--accent);
```

FROM `apps/web/src/tokens.css`

```
:root[data-theme="dark"] {
  color-scheme: dark;

  --bg: #0f1216;
  --surface: #171b21;
  --surface-2: #21262e;
  --canvas: #13161b;
  --canvas-dot: #2f3740;
  --border: #2c333d;
  --border-strong: #7b8694;
  --frame-shadow: 0 2px 8px rgb(0 0 0 / 0.6);
```

Two decisions here. **One accent.** Selection, focus ring, links and the primary button share `--accent` ; Figma's discipline, and it makes "what is interactive" one colour a person learns once. **The dark block keys on an attribute, not a media query.** `html[data-theme]` carries the *resolved* theme; a media query would need a second copy of every token for the "OS dark but the person chose light" case, and would flash the OS theme before script ran. `color-scheme` tells the browser which palette its own form controls and scrollbars should use, so a `<select>` in the dark theme is dark too.

FROM `apps/web/src/theme.ts`

```
export const resolveTheme = (preference: ThemePreference, systemDark: boolean): Theme => (preference === "system" ?
(systemDark ? "dark" : "light") : preference);

/** What the toggle remembers: the OPPOSITE of what is shown, as an explicit choice, so it holds when the OS changes its mind.
*/
export const toggled = (shown: Theme): Theme => (shown === "dark" ? "light" : "dark");
```

FROM `apps/web/src/useTheme.ts`

```
export function useTheme(): { theme: Theme; toggle: () => void } {
  const [preference, setPreference] = useState<ThemePreference>(remembered);
  const dark = useSyncExternalStore(onSystemChange, systemDark, () => false);
  const theme = resolveTheme(preference, dark);
  useEffect(() => { apply(theme); }, [theme]);
  return {
    theme,
    toggle: () => { const next = toggled(theme); remember(next); setPreference(next); },
  };
}
```

`theme.ts` is pure: a remembered preference and the OS's answer come in, a theme comes out, and `toggled` remembers the *opposite of what is shown* as an explicit choice, so a person who pressed the toggle on a dark OS gets light and keeps light when the OS flips.

`useTheme` binds it: the OS preference is an external store (a `matchMedia` change is not a render), read through `useSyncExternalStore` exactly as the peer is; the choice is `useState` initialised from `localStorage`, read defensively (a private window may refuse it); and an effect writes the resolved theme onto `<html>`. `main.tsx` calls `applyStoredTheme()` before the first render, so the first paint is already right: no flash.

FROM `apps/web/src/theme.test.ts`

```
const light = tokensOf(":root {}");
// The dark block only overrides colours: what it does not name is inherited from the light block.
const dark = new Map([...light, ...tokensOf(':root[data-theme="dark"]')]);

// [text, on surface, minimum]: 4.5:1 for text, 3:1 for borders, focus rings and the canvas dots' frame.
const PAIRS: [string, string, number][] = [
  ["fg", "surface", 4.5], ["fg", "bg", 4.5], ["fg", "canvas", 4.5], ["fg", "surface-2", 4.5], ["fg", "accent-soft", 4.5],
  ["fg-muted", "surface", 4.5], ["fg-muted", "bg", 4.5], ["fg-muted", "canvas", 4.5],
  ["accent", "surface", 4.5], ["accent", "bg", 4.5], ["accent", "canvas", 4.5], ["on-accent", "accent", 4.5],
  ["danger-fg", "danger-bg", 4.5], ["warning-fg", "warning-bg", 4.5], ["info-fg", "info-bg", 4.5],
  ["border-strong", "surface", 3], ["border-strong", "canvas", 3], ["accent", "surface-2", 3],
  ["danger-border", "danger-bg", 3], ["warning-border", "warning-bg", 3], ["info-border", "info-bg", 3],
];

for (const [name, tokens] of [["light", light], ["dark", dark]] as const) {
  test(`${name} tokens: every text colour reads AA on the surface it sits on, borders and rings at 3:1`, () => {
    expect(tokens.size).toBeGreaterThan(20);
    for (const [text, on, minimum] of PAIRS) {
      const [a, b] = [tokens.get(text), tokens.get(on)];
      if (a === undefined || b === undefined) throw new Error(`${name} theme is missing --${text} or --${on}`);
      expect(contrast(a, b), `${name}: --${text} on --${on}`).toBeGreaterThanOrEqual(minimum);
    }
  });
}
```

The contrast test is the part to remember for an interview: *accessibility as a unit test*. It parses `tokens.css` itself (not a copy of the values), builds the dark palette as the light one overridden by the dark block (what the cascade does), and recomputes WCAG 2.x relative luminance and contrast (`colour.ts`, the same function that checks the presence palette) for every text/surface pair the UI actually uses, at 4.5:1 for text and 3:1 for borders and rings. A token edit that breaks contrast is a red test, not a review comment. The last test in the file

asks that the dark block redefine every colour the light block declares, because a colour left out would inherit its light value and show light on dark.

FROM `apps/web/src/app.css`

```
/* The editor's own control rules ('input', 'button' above) reach into the frame and the tiles too; there they resolve to
   the design system's palette, so a field on the canvas is the sample app's field in the dark theme as well. */
.page-frame, .library .thumb-frame { --surface: var(--ds-surface); --surface-2: var(--ds-ground); --fg: var(--ds-ink); --fg-
muted: var(--ds-muted); --border: var(--ds-line); --border-strong: var(--ds-line); }
```

And the cascade's second use: *the frame is the sample app's page, not the editor's*. The design system's own tokens (`--ds-*`) apply inside the frame and the library tiles; so that the editor's own `input` and `button` rules, which reach into the frame too, do not paint the sample app's field with the dark theme's surface, the editor's tokens are *redefined* on `.page-frame` to resolve to the design system's palette. One rule, and a field on the canvas is the sample app's field in both themes (E10.6's hand-off; the cursors spec asserts the Input is white in dark).

TYPESCRIPT + REACT (THIS REPO)	JAVA (SWING, JAVAFX)	PYTHON (QT, TK)
<code>--accent</code> on <code>:root</code> , <code>var(--accent)</code> below, resolved per element at style time	<code>UIManager.put("Panel.background", ...)</code> and a Look-and-Feel	a Qt stylesheet with <code>palette(...)</code> ; Tk's <code>option_add</code>
<code>:root[data-theme="dark"] { --bg: ... }:</code> one attribute flips everything	swap the LookAndFeel and <code>updateComponentTreeUI</code>	<code>app.setPalette(dark)</code>
<code>color-scheme: dark</code> for the browser's own controls	n/a: the L&F draws them	the palette covers them
contrast recomputed in a unit test from the CSS	a manual audit	a manual audit

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

The presence palette was checked on white, not on a dark surface.

E2.6 derived a colour per peer and checked it read as text on white. In the dark theme those names sat on a dark surface at about 3:1. The fix was not a second palette: the colour became an avatar's *fill* with white initials (the palette is checked to carry white text, `colour.test`), and the name beside it is plain `--fg`. A colour that has to work as text on two backgrounds is a colour that has to be tested twice; a colour that is only ever a background for white text is tested once.

3 · Coordinate systems: one transform, outlines in world space, and the deepest box wins

An infinite canvas has two coordinate systems. *World*: the page frame's own pixels at 100 %, where the Button is at (240, 100) for ever. *Screen*: where a world point is drawn inside the canvas element right now, given the zoom and the pan. One convention everywhere, and the whole of `viewport.ts` follows from it:

FROM `apps/web/src/viewport.ts`

```
export const toWorld = (v: Viewport, screen: Point): Point => ({ x: (screen.x - v.x) / v.zoom, y: (screen.y - v.y) / v.zoom
});
export const toScreen = (v: Viewport, world: Point): Point => ({ x: world.x * v.zoom + v.x, y: world.y * v.zoom + v.y });

/** Zooms to `zoom` (clamped) so that the world point under `anchor` (a screen point: the pointer, the view's centre) stays
under it. */
export function zoomAt(v: Viewport, zoom: number, anchor: Point): Viewport {
  const next = clampZoom(zoom);
  const held = toWorld(v, anchor);
  return { zoom: next, x: anchor.x - held.x * next, y: anchor.y - held.y * next };
}
```

`zoomAt` is the function a beginner gets wrong: zooming about the pointer means the world point under the pointer must stay under it, so take that point (`toWorld` at the old zoom), then choose the new offset so `toScreen` of it at the new zoom is the anchor again. The test asserts exactly this for four zooms, and that a point elsewhere on the screen moves away from the anchor by the right factor. Everything is pure, clamped to 10–400 %, and tested as arithmetic; `Surface` only wires events to it.

FROM `apps/web/src/viewport.ts`

```
export function fit(view: Size, content: Size, padding = 32): Viewport {
  const zoom = clampZoom(Math.min((view.width - 2 * padding) / content.width, (view.height - 2 * padding) / content.height));
  return { zoom, x: (view.width - content.width * zoom) / 2, y: (view.height - content.height * zoom) / 2 };
}

/** The viewport that shows 'world' in the middle of the view, at the zoom it has (E10.6: jumping to someone's selection). */
export const centreOn = (v: Viewport, view: Size, world: Point): Viewport => ({ zoom: v.zoom, x: view.width / 2 - world.x *
v.zoom, y: view.height / 2 - world.y * v.zoom });
```

FROM `apps/web/src/Surface.tsx`

```
<div className="world" data-zoom={viewport.zoom.toFixed(2)} style={{ transform: `translate(${String(viewport.x)}px,
${String(viewport.y)}px) scale(${String(viewport.zoom)})`, "--zoom": viewport.zoom } as CSSProperties}>
```

FROM `apps/web/src/app.css`

```
/* ONE transform for everything on the sheet; strokes and labels below divide by --zoom to stay the same size on the screen.
*/
.world { position: absolute; left: 0; top: 0; transform-origin: 0 0; }
/* The frame keeps the sample app's own look (its stylesheet is scoped to it, designSystem.ts) in both editor themes, like a
white sheet on a dark desk.
One desktop width here: the device frames are the preview's (E10.7), where the page runs for real. */
```

The frame and the editor layer (outlines, cursors, shades, drop lines) both live in `.world`, which carries the *one* `translate(x, y) scale(zoom)`. Zooming and panning therefore change one style attribute; nothing is re-laid out, nothing is re-measured, and the GPU composites the layer. The price is that strokes and labels would scale with the frame, so they divide by `--zoom` in CSS:

FROM `apps/web/src/app.css`

```
/* A box-shadow spread, not a border: Chrome snaps border widths to whole pixels, which would thicken the stroke at high zoom.
*/
.outline { position: absolute; --stroke-colour: var(--accent); box-shadow: 0 0 0 calc(2px / var(--zoom)) var(--stroke-colour);
}
.outline-hovered { box-shadow: 0 0 0 calc(1px / var(--zoom)) var(--stroke-colour); }
```

A 2 px stroke at 300 % is `calc(2px / 3)` in world units, which is 2 px on the screen. The stroke is a `box-shadow` spread and not a `border` because Chrome snaps border widths to whole device pixels, so a border thickened in steps as the zoom rose; a shadow spread stays fractional. The labels do the same for their padding, radius and font size.

FROM `apps/web/src/Surface.tsx`

```
// The outlines follow the components' real boxes, measured after every render and kept in WORLD
// coordinates, so zooming and panning move them by CSS alone. Set only when something moved: a
// measure that finds the same boxes must not render again.
useLayoutEffect(() => {
  const el = view.current;
  if (!el) return;
  const box = el.getBoundingClientRect();
  const wanted: Pick<Outline, "id" | "kind" | "label" | "colour">[] = [
    { id: selected, kind: "selected", label: labels.get(selected) ?? "", },
    ...(hovered !== null && hovered !== selected ? [{ id: hovered, kind: "hovered" as const, label: labels.get(hovered) ??
"" }] : []),
    ...[...selectedBy].flatMap(([id, peers]) => peers.map((p) => ({ id, kind: "peer" as const, label: nameOf(p), colour:
colourOf(p.peerId) }))),
  ];
  const toOutline = (w: Pick<Outline, "id" | "kind" | "label" | "colour">, rect: Rect): Outline => {
    const at = toWorld(viewport, { x: rect.left - box.left, y: rect.top - box.top });
    return { ...w, x: hundredths(at.x), y: hundredths(at.y), width: hundredths((rect.right - rect.left) / viewport.zoom),
height: hundredths((rect.bottom - rect.top) / viewport.zoom) };
  };
  setOutlines((current) =>
```

The outlines are *measured*, not computed: the real components lay themselves out (a Stack's flex row decides where its Button is), so after each render a `useLayoutEffect` reads every selected, hovered and peer-selected node's `getBoundingClientRect()` and converts it to world coordinates by `toWorld` and a division by the zoom. `useLayoutEffect` rather than `useEffect` because it runs synchronously after the DOM is updated and before the browser paints, so the outline never lags the frame by a frame. Once in world coordinates the outline zooms and pans by CSS alone: the measuring pass runs after a *render*, never after a wheel event. The `setOutlines((current) =>`

same ? current : next) shape matters: an effect that sets state on every run would render again for ever; returning the same object when nothing moved ends the loop.

FROM `apps/web/src/Surface.tsx`

```
// A native listener, not onWheel: it must preventDefault (ctrl+wheel is otherwise the browser's own
// zoom, plain wheel the pane's scroll), and React does not promise a non-passive wheel listener.
useEffect(() => {
  const el = view.current;
  if (!el) return;
  const onWheel = (event: WheelEvent): void => {
    event.preventDefault();
    const box = el.getBoundingClientRect();
    if (event.ctrlKey || event.metaKey) setViewport((v) => zoomAt(v, wheelZoom(v.zoom, event.deltaY), { x: event.clientX -
box.left, y: event.clientY - box.top }));
    else setViewport((v) => panBy(v, -event.deltaX, -event.deltaY));
  };
  el.addEventListener("wheel", onWheel, { passive: false });
  return () => { el.removeEventListener("wheel", onWheel); };
}, []);
```

One place React's abstraction is not enough: the wheel. Ctrl+wheel is the browser's own page zoom and a plain wheel scrolls the pane, so the handler must call `preventDefault()`, which is only allowed on a listener registered with `{ passive: false }`; React does not promise that for `onWheel`. So the listener is native, added in an effect, removed in its cleanup. Trackpad pinches arrive as many small `deltaY`s, a mouse notch as about 100; `wheelZoom` is exponential in the delta, so up-then-down lands where it started, and capped so one wild notch cannot jump the range.

FROM `apps/web/src/selection.ts`

```
export function hitTest(boxes: readonly Box[], point: Point): string | null {
  let hit: Box | null = null;
  for (const box of boxes) {
    if (point.x < box.left || point.x >= box.right || point.y < box.top || point.y >= box.bottom) continue;
    if (!hit || box.depth >= hit.depth) hit = box;
  }
  return hit?.id ?? null;
}
```

Hit-testing does not ask the DOM which element is under the pointer (`elementFromPoint` would answer with a `<span>` inside the Button, or the frame), it asks the *document*: every node's box is measured, and the deepest box containing the point wins, because a child is drawn on top of its parent and is what the eye lands on; among equals the later in document order wins, which is the one painted last. Nothing under the point is the empty sheet, and the caller selects the page. Pure, so tested with plain boxes.

FROM `apps/web/src/Surface.tsx`

```
function NodeView({ doc, node, depth, selectedBy, isContainer }: { doc: Doc; node: DocNode; depth: number; selectedBy:
Map<string, Presence[]>; isContainer: (id: string) => boolean }) {
  const Component = components[node.component];
  const children = node.children.flatMap((id) => { const child = doc.nodes[id]; return child ? [<NodeView key={id} doc={doc}
node={child} depth={depth + 1} selectedBy={selectedBy} isContainer={isContainer} />] : []; });
  return (
    <div data-node-id={node.id} data-component={node.component} data-depth={depth} data-empty-container={children.length === 0
&& isContainer(node.id) ? "" : undefined} className="node" {...selectedByAttr(selectedBy.get(node.id) ?? [])}>
      {Component ? createElement(Component, node.props, children) : <div className="unknown-component">{node.component}</div>}
      <span className="node-props">{propsText(node)}</span>
    </div>
  );
}
```

FROM `apps/web/src/designSystem.ts`

```
export const scopeTo = (css: string, className: string): string => css.replace(/(^|[\s])(?:::root|body)(?=\s*\{)/gu,
`$1.${className}`);
export const scopeToFrame = (css: string): string => scopeTo(css, FRAME);
```

The components are the sample app's real ones, imported straight from the seed (they are the repo's own static code, the same files the manifest was extracted from), with each node's children rendered *inside* its component, so a Stack lays out its children exactly as the running page would. The wrapper is `display: contents`: it names the node for the editor and takes no box of its own, so the flex items

are the components themselves. The frame is `inert`, an HTML attribute that makes a subtree take no clicks, no focus and no screen-reader stop: the Button *is* the sample app's Button, but pressing it does nothing, and the canvas is the one focusable thing. The seed styles its page on `:root` and `body`; on the canvas the page is one frame among the editor's surfaces, so those two selectors are rewritten to the frame's class and everything else is left as written: one source of truth, the seed's own stylesheet, scoped on load. The `ponytail` is honest about the ceiling: a regex over CSS the repo owns; a customer's design system would render in a sandboxed iframe, never in the editor's origin.

TYPESCRIPT + REACT (THIS REPO)	JAVA (SWING, JAVAFX)	PYTHON (QT, TK)
<code>transform: translate()</code> <code>scale()</code> on one layer; strokes divide by <code>--zoom</code>	<code>Graphics2D.transform(AffineTransform)</code> in <code>paintComponent</code> ; strokes divided by the scale by hand	<code>QGraphicsView.setTransform</code> ; <code>QPen.setCosmetic(True)</code> for a constant-width stroke
<code>toWorld</code> / <code>toScreen</code> as two pure functions	<code>AffineTransform.inverseTransform</code>	<code>QGraphicsView.mapToScene</code> / <code>mapFromScene</code>
the deepest measured box wins	<code>Component.getComponentAt</code> (the shallowest)	<code>QGraphicsScene.itemAt</code> (the topmost by z)
<code>inert</code> on the frame	a glass pane over the components	<code>setAttribute(Qt.WA_TransparentForMouseEvents)</code>
measure after render in <code>useLayoutEffect</code>	<code>getBounds()</code> after <code>validate()</code>	<code>sceneBoundingRect()</code> after layout

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

An outline drawn as a border thickened with the zoom.

The first outline was `border: 2px solid` divided by `--zoom`. At 250 % it was visibly thicker than at 100 %, and at 40 % it vanished: Chrome snaps border widths to whole device pixels before compositing. The e2e spec (`canvas-zoom-pan-select`) asserts a constant stroke width across zooms; the fix was a box-shadow spread, which stays fractional. A learning about the renderer, not about React, and the kind a screenshot baseline catches and a unit test cannot.

4 · The layers tree: an ARIA tree whose selection is the canvas's, and a drop that is one `move_node`

The tree is the same document, rendered a second time, and the point of the section is what that costs: nothing. `LayersPanel` receives `doc`, the rows `Canvas.tsx` names ("Stack 2" is the second Stack in reading order), `selected` and `submit`; picking in the tree calls `onSelect`, which sets `wanted` in `Canvas`, which re-renders both the tree and the canvas with the new selection. There is no "sync the tree to the canvas" code because there are not two selections.

FROM `apps/web/src/LayersPanel.tsx`

```
<li
  key={row.id}
  role="treeitem"
  aria-level={row.depth + 1}
  aria-setsize={siblings.length}
  aria-posinset={siblings.indexOf(row.id) + 1}
  aria-selected={row.id === selected}
  {...(hasChildren(row.id) ? { "aria-expanded": !collapsed.has(row.id) } : {})}
  tabIndex={row.id === tabStop ? 0 : -1}
  data-node-id={row.id}
  data-drop={over ? (over.allowed ? over.placement : "none") : undefined}
  data-dragging={drag?.nodeId === row.id ? "" : undefined}
  className="tree-row"
  style={{ "--depth": row.depth } as CSSProperties}
  onPointerDown={event => { onPointerDown(event, row.id); }}
>
```

The WAI-ARIA Authoring Practices *tree* pattern, in its flat form: one `role=tree`, every row a `treeitem` with `aria-level`, `aria-setsize` and `aria-posinset` (so a screen reader says "Button 1, level 3, 1 of 2"), `aria-expanded` only on rows that have children,

`aria-selected` on the one selected. The APG allows a flat list when nesting the DOM would only nest the problems, and here it makes the drop hit-test and the roving tabindex one `querySelector` each. *Roving tabindex*: exactly one row has `tabIndex=0` (the selected row, or its nearest shown ancestor when it is folded away), every other row `-1`, so Tab enters the tree once and arrows move inside it; that is the pattern for every composite widget in this epic (the tree, the library's listbox). The disclosure triangle is a `span`, not a `button`, because a control inside a `treeitem` is "nested interactive" and axe flags it; Left and Right are its keys.

FROM `apps/web/src/LayersPanel.tsx`

```
// The tab stop: the selected row, or the nearest shown row above it when it is folded away.
let tabStop = selected;
for (let guard = 0; !shownIds.has(tabStop) && guard < rows.length; guard++) tabStop = doc.nodes[tabStop]?.parentId ??
doc.rootId;
```

FROM `apps/web/src/layer-moves.ts`

```
export function dropToMoveOp(doc: Doc, drop: Drop): MoveOp | null {
  const node = doc.nodes[drop.nodeId];
  const target = doc.nodes[drop.targetId];
  if (!node || !target || node.parentId === null) return null;
  if (within(doc, drop.targetId, drop.nodeId)) return null; // onto itself, or into its own subtree
  if (drop.placement === "into") return { type: "move_node", nodeId: node.id, newParentId: target.id, index:
target.children.filter((id) => id !== node.id).length };
  if (target.parentId === null) return null; // beside the page: there is no such place
  const parent = doc.nodes[target.parentId];
  if (!parent) return null;
  const siblings = parent.children.filter((id) => id !== node.id);
  return { type: "move_node", nodeId: node.id, newParentId: parent.id, index: siblings.indexOf(target.id) + (drop.placement
=== "after" ? 1 : 0) };
}
```

This is the function the chapter is named for. A `Drop` (which node, over which row, before/after/into) becomes *one* `move_node`, or null when there is nowhere to go. Two details are load-bearing. **The index is the node's FINAL position among its new siblings, counted after it was taken out** (SPEC keystone 3): dropping `card` after `stack` in `[card, stack, text2]` means `[stack, card, text2]`, index 1, not 2, which is why the siblings are filtered before `indexOf`. Get this wrong and every downward move within a parent lands one too far; the drill in this lesson plants exactly that. **It refuses only what is structurally impossible on the document it can see**: the page itself, a node onto itself or into its own subtree (a cycle; `within` carries a `seen` set so a corrupt document with a cycle cannot hang the tab), a place beside the page. A container that takes no children, an index past the end, a node someone removed meanwhile: those are the replica's and the room's verdict, shown as their refusal like any other edit's, because a second copy of `validate()` here would drift.

FROM `apps/web/src/layer-moves.ts`

```
export function placementAt(fraction: number, container: boolean): Placement {
  if (!container) return fraction < 0.5 ? "before" : "after";
  return fraction < 0.25 ? "before" : fraction >= 0.75 ? "after" : "into";
}
```

FROM `apps/web/src/LayersPanel.tsx`

```
// --- the pointer: press selects; a press that travels becomes a drag -----
const onPointerDown = (event: ReactPointerEvent<HTMLLIElement>, id: string): void => {
  if (event.button !== 0) return;
  onSelect(id);
  if (id !== doc.rootId) press.current = { id, x: event.clientX, y: event.clientY };
};
const onPointerMove = (event: ReactPointerEvent<HTMLUListElement>): void => {
  const p = press.current;
  if (!p) return;
  if (!drag) {
    if (Math.hypot(event.clientX - p.x, event.clientY - p.y) < 4) return;
    try { event.currentTarget.setPointerCapture(event.pointerId); } catch { /* a pointer the browser no longer tracks: the
drag still works while it stays over the tree */ }
  }
  const row = rowUnder(event.clientX, event.clientY);
  const targetId = row?.dataset["nodeId"];
  let over: Drag["over"] = null;
  if (row && targetId !== undefined) {
    const box = row.getBoundingClientRect();
    const placement = placementAt((event.clientY - box.top) / box.height, isContainer(targetId));
```

```

const drop = { nodeId: p.id, targetId, placement };
over = { ...drop, allowed: dropToMoveOp(doc, drop) !== null };
}
const next = { nodeId: p.id, over };
setDrag((was) => (JSON.stringify(was) === JSON.stringify(next) ? was : next));
};
const endDrag = (): void => { press.current = null; setDrag(null); };
const onPointerUp = (): void => {
  if (drag?.over?.allowed) move(dropToMoveOp(doc, drag.over));
  endDrag();
};

```

The drag is *pointer events*, not HTML drag and drop. A press selects and remembers where; a press that travels 4 px becomes a drag and *captures* the pointer (`setPointerCapture` : every further event comes to the tree whatever is under the pointer, so letting go over the canvas still ends the drag cleanly); on each move the row under the pointer is found with `elementFromPoint` (here the DOM is right, because the rows are the DOM), the pointer's height on the row becomes a placement (`placementAt` : a container's middle half is "into"; a leaf has only a top and a bottom half, so the line always shows where the node *would* go and "into" is never offered), and the row wears a line or a box; `pointerup` sends the one op. HTML drag and drop was rejected because the canvas already speaks pointer events (one gesture vocabulary), it draws a ghost image to fight, and its `dragover` fires on its own clock. The `setDrag((was) => same ? was : next)` shape is the outline trick again: no render for a move that changed nothing.

The keyboard's four moves are drops too: Alt+Up is "before my previous sibling", Alt+Down "after my next", Alt+Right "into my previous sibling" (the room says whether it takes children), Alt+Left "after my parent"; `keyMoveOp` builds the `Drop` and calls the same `dropToMoveOp` , so the pointer and the keys cannot disagree about the op. Delete sends `remove_node` , and the focus falls to the page with the selection. A live region says every move and removal in words, because a moved row is silent to a screen reader.

TYPESCRIPT + REACT (THIS REPO)	JAVA (SWING, JAVAFX)	PYTHON (QT, TK)
<code>role=tree</code> , flat <code>treeitem</code> s with <code>aria-level</code> , roving tabindex	<code>JTree</code> with a <code>TreeModel</code> ; focus handled by the L&F	<code>QTreeView</code> over a <code>QAbstractItemModel</code>
the tree renders <code>peer.doc</code> ; no model of its own	<code>DefaultTreeModel</code> mirrors your data and you keep them in step	the model IS your data through <code>data()</code> / <code>index()</code>
a drop is one <code>move_node</code> through <code>submit</code>	<code>TransferHandler.importData</code> mutates the model	<code>dropMimeData</code> mutates the model
pointer events + <code>setPointerCapture</code>	<code>DragSource</code> / <code>DropTarget</code> (AWT DnD)	<code>QDrag.exec()</code>
refuse only geometry; the replica and the room say the rest	<code>canImport</code> predicts the rule	<code>canDropMimeData</code> predicts the rule

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

A drop into a leaf was disabled in the tree, and the tree learnt a rule it did not own.

The first draft greyed out a Button row while a layer was dragged over it, "so the person never sees a refusal". Review asked where the tree learnt that a Button takes no children: from the manifest, a second reading of a rule `validate()` already applies. The bead's own text settled it: the tree refuses only what the document's shape makes impossible; everything else is built and refused by the replica in the same frame, with the same sentence a viewer, a slow room or the AI would get. The e2e spec asserts the sentence for Alt+Right into a Button rather than a colour on a row.

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

Selecting on focus would have moved the selection when Tab landed on the tree.

The APG's tree pattern lets focus and selection follow each other, and the first version selected whatever row received focus. But the tab stop is the selected row *or its nearest shown ancestor* when the selection is folded away; Tab landing on that ancestor would have changed the selection to it. Now arrows move focus and selection together (`select`), but a bare focus (Tab) changes nothing, and Space or Enter selects the focused row.

5 · The inspector is the manifest, rendered: one control per prop kind, one op per edit

The inspector knows no component by name. The manifest (lesson 2: extracted from the design system's source) says, for each component, which props it has, of what kind, whether required, with what default; the inspector renders *that*. So a regenerated manifest with a new prop gets its control with no code change, and `controls.test` walks the committed manifest to prove every prop has one.

FROM `apps/web/src/controls.ts`

```
export function controlFor(prop: Prop): Control {
  const placeholder = prop.default === undefined ? (prop.required ? "required" : "") : String(prop.default);
  const type = prop.type;
  switch (type.kind) {
    case "string":
      return { kind: "text", prop, placeholder };
    case "number":
      return { kind: "number", prop, placeholder };
    case "boolean":
      return { kind: "toggle", prop, fallback: prop.default === true };
    case "enum": {
      const fallback = typeof prop.default === "string" ? prop.default : undefined;
      return { kind: type.options.length <= SEGMENTED_MAX ? "segmented" : "select", prop, options: type.options, fallback };
    }
    default: {
      // The compiler's check that every kind above is handled, and the runtime's answer when a manifest
      // from a newer extractor arrives with a kind this build has never heard of.
      const unknown: never = type;
      throw new Error(`no control for prop kind ${JSON.stringify((unknown as { kind?: unknown }).kind)} (${prop.name})`);
    }
  }
}
```

Read the `default` branch twice. `const unknown: never = type` is the compiler's exhaustiveness check: `type` is a discriminated union of four kinds, and after four `case` s TypeScript narrows it to `never`; add a fifth kind to the contract and this line stops compiling until a control exists for it. The `throw` under it is the runtime's answer to a manifest from a newer extractor that this build has never heard of: a prop without a control must fail loudly, because a prop that renders as nothing is a prop nobody can edit and nobody notices. Java has this as a `switch` over a sealed interface with pattern matching (Java 21); Python has `assert_never` from `typing`.

FROM `apps/web/src/controls.ts`

```
const LAYOUT_PROPS: Readonly<Record<LayoutName, Prop["type"]["kind"]>> = { direction: "enum", gap: "number", padding: "number", align: "enum" };
/** The layout props that stand for a SPACE the canvas can shade while their control is hovered or focused. */
export type Space = Extract<LayoutName, "gap" | "padding">;

const isLayoutName = (name: string): name is LayoutName => Object.hasOwn(LAYOUT_PROPS, name);
const layoutNameOf = (prop: Prop): LayoutName | undefined => (isLayoutName(prop.name) && LAYOUT_PROPS[prop.name] === prop.type.kind ? prop.name : undefined);
export const isLayoutProp = (prop: Prop): boolean => layoutNameOf(prop) !== undefined;
export const spaceOf = (prop: Prop): Space | undefined => { const name = layoutNameOf(prop); return name === "gap" || name === "padding" ? name : undefined; };
```

The Layout section is the one thing keyed on a name, and deliberately on *name and kind*: `gap` as a number is a stepper that shades the space between children; `gap` as a string is just a string and lands in Props. A design system whose Stack calls it `spacing` gets a plain stepper and no shade, which is correct: the shade would have nothing true to show.

FROM `apps/web/src/Inspector.tsx`

```
function NumberField({ control, value, ids, onChange }: { control: Extract<Control, { kind: "number" }>; value: Value; ids: Ids; onChange: OnChange }) {
  const [draft, setDraft] = useState<string>();
  const name = control.prop.name;
  const resting = typeof value === "number" ? value : typeof control.prop.default === "number" ? control.prop.default : 0;
  const shown = draft !== undefined && draft !== "" && Number.isFinite(Number(draft)) ? Number(draft) : resting;
  const stepBy = (direction: 1 | -1, big: boolean): void => { setDraft(undefined); onChange(plain(shown + direction * (big ? 10 : 1))); };
  const commit = (input: HTMLInputElement): void => {
    if (draft === undefined) return;
    setDraft(undefined);
    // A number field reports "" BOTH when it is empty and when it holds text it cannot read ("1e999",
    // "12abc"). Only the browser knows which: validity.badInput. Unreadable = no edit; taking it for ""
  }
}
```

```

    // would silently delete the prop.
    if (input.validity.badInput) return;
    if (draft === "") onChange(null);
    else if (Number.isFinite(Number(draft))) onChange(plain(Number(draft)));
  };
  // The keys are the registry's (shortcuts.ts, scope "inspector"); which way a step goes, and how far, the event still says.
  const onKeyDown = (event: ReactKeyboardEvent<HTMLInputElement>): void => {
    const action = actionFor("inspector", event);
    if (action === "commit") commit(event.currentTarget);
    else if (action === "step") { event.preventDefault(); stepBy(event.key === "ArrowUp" ? 1 : -1, event.shiftKey); }
  };
  const press = (direction: 1 | -1) => (event: ReactMouseEvent): void => { stepBy(direction, event.shiftKey); };
  return (
    <span className="stepper">
      <button type="button" tabIndex={-1} aria-label={`Decrease ${name}`} onClick={press(-1)}></button>
      <input id={ids.id} type="number" className="mono" aria-describedby={ids.describedBy} placeholder={control.placeholder}
value={draft ?? (typeof value === "number" ? String(value) : "")} onChange={event => { setDraft(event.target.value); }}
onBlur={event => { commit(event.currentTarget); }} onKeyDown={onKeyDown} />
      <button type="button" tabIndex={-1} aria-label={`Increase ${name}`} onClick={press(1)}></button>
    </span>
  );
}

```

The commit semantics are E2.5b's, kept on purpose. A *typed* value (text or number) is one `set_prop` when the person leaves the field or presses Enter, never one per keystroke: a flood of ops would meet the room's per-peer rate limit and fill the audit log with "title = H", "title = He", "title = Hel". The draft lives in `useState` only while the field is being edited; the document stays the source of truth, and `value={draft ?? prop}` means the field shows the document the moment the draft is committed or abandoned. A choice, a switch, an arrow step and a stepper button are each one deliberate edit, so they send at once. The `validity.badInput` line is a browser fact worth knowing: a number field reports `" "` both when it is empty and when it holds text it cannot read ("12abc"); taking the second for the first would silently delete the prop.

FROM `apps/web/src/Inspector.tsx`

```

<div className="field" data-prop={prop.name} data-refused={refusal === undefined ? undefined : ""}>
  {grouped ? <span id={nameId} className="field-name">{prop.name}</span> : <label htmlFor={id}>{prop.name}</label>}
  <span className="field-control">
    {control.kind === "text" ? <TextField control={control} value={value} ids={ids} onChange={onChange} />
      : control.kind === "number" ? <NumberField control={control} value={value} ids={ids} onChange={onChange} />
      : control.kind === "toggle" ? <Toggle control={control} value={value} ids={ids} onChange={onChange} />
      : control.kind === "segmented" ? <Segmented control={control} value={value} ids={{ ...ids, id: nameId }}
onChange={onChange} />
      : <Select control={control} value={value} ids={ids} onChange={onChange} />}
  </span>
  {isSet && !prop.required ? <button type="button" className="reset" aria-label={`Reset ${prop.name}`} onClick={() => {
onChange(null); }}>Reset</button> : <span className="reset-space" />}
  {!isSet && (control.kind === "toggle" || control.kind === "segmented") && prop.default !== undefined && <span
className="hint field-default">default {String(prop.default)}</span>}
  {refusal !== undefined && <span id={refusalId} className="field-refusal">{sentenceFor(refusal)}</span>}
</div>
);
}

```

One row per prop: the name as a `label` (or, for a radio group, a span the group is `aria-labelledby`, because a `fieldset` that has been given `role=radiogroup` is not reliably named by its `legend`), the control, Reset while the prop is set (sending `set_prop null`, which removes it; not offered for a required prop, which has no unset state), the default in words while it is not, and the refusal its last edit met. `aria-describedby` ties the refusal sentence to the control so a screen reader hears it when the field is focused; there is deliberately no second `role=alert` here, because the notice strip already announced it. `usePeer`'s `Refusal` remembers its `op` for exactly this: the inspector filters the refusals to the selected node's `set_prop`s, latest per key, and `submit` dismisses the old complaint about a prop before a new edit of it is sent, so a fixed value clears both.

FROM `apps/web/src/spaces.ts`

```

export function gapsBetween(children: readonly Rect[]): Rect[] {
  const gaps: Rect[] = [];
  for (let i = 1; i < children.length; i++) {
    const a = children[i - 1];
    const b = children[i];
    if (!a || !b) continue;
    if (b.left > a.right) gaps.push({ left: a.right, top: Math.min(a.top, b.top), right: b.left, bottom: Math.max(a.bottom, b.bottom) });
  }
}

```

```

    else if (b.top > a.bottom) gaps.push({ left: Math.min(a.left, b.left), top: a.bottom, right: Math.max(a.right, b.right),
bottom: b.top });
    }
    return gaps;
  }

```

FROM `apps/web/src/Surface.tsx`

```

function paddingOf(own: Element, zoom: number): Sides {
  const style = getComputedStyle(own);
  const px = (value: string): number => (Number.parseFloat(value) || 0) * zoom;
  return { top: px(style.paddingTop), right: px(style.paddingRight), bottom: px(style.paddingBottom), left:
px(style.paddingLeft) };
}

```

Shading is the inspector talking to the canvas through `Canvas` 's state: hovering *or focusing* a gap or padding control sets a `Hint` ; `Surface` measures that node (the ring of computed padding inside its box, or the spaces between consecutive children's boxes; `spaces.ts` is pure geometry) in world coordinates and draws translucent outlines, so the shade zooms with the frame. Padding is read from the component's *computed* style, what the design system actually applied whether from the prop or its own default, and in the element's own px times the zoom, because `getBoundingClientRect` answers in screen px after the world's scale and `getComputedStyle` in CSS px before it.

TYPESCRIPT + REACT (THIS REPO)

JAVA (SWING, JAVAFX)

PYTHON (QT, TK)

`controlFor(prop)` : a switch over the prop's kind, exhaustive by `never`

a `PropertyEditor` per type, looked up by `PropertyEditorManager`

`QItemDelegate.createEditor` by `QVariant` type

commit on blur or Enter; one op

`JFormattedTextField` commits on focus lost

`QLineEdit.editingFinished`

`value={draft ?? prop}` : the document is the source, the draft a moment

the text field IS the state until you read it back

the widget IS the state

`set_prop null` removes a prop; a required one has no Reset

a "restore default" per editor

the same

`aria-describedby` the refusal

a tooltip on the field

`setToolTip`

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

A radio group's legend did not name it once it had a role.

The segmented control is a `fieldset` with `role=radiogroup` and had a `legend` for its name. axe reported the group unnamed in one engine: a `fieldset` given an explicit role is no longer a fieldset to the accessibility tree, and its legend no longer counts. The name is now a visible span the group is `aria-labelledby` . The same file carries a second lesson about native controls: native `<input type=number>` spin arrows were rejected because they cannot do Shift ×10 and, worse, do not commit, so a spin would have changed the field and sent nothing.

6 · The library: three intents, one Slot, one add_node, an id minted once

The "Add X" toolbar becomes a listbox of tiles, each the real component rendered small under a second scoping of the seed's stylesheet, `inert` and `aria-hidden` (a tile's name is the component's, not its sample text). Three gestures add: Enter or Space on a tile, a click, and a drag onto a layer row or onto the canvas. All three end in a `Slot` , and a slot becomes the op.

FROM `apps/web/src/library-adds.ts`

```

export function addOpAt(slot: Slot, component: Component, nodeId: string): AddOp {
  return { type: "add_node", nodeId, parentId: slot.parentId, index: slot.index, component: component.name, props:
requiredProps(component) };
}

```

FROM `apps/web/src/library-adds.ts`

```
export function slotOnCanvas(doc: Doc, hit: string | null, isContainer: (id: string) => boolean, indexIn: (containerId: string) => number): Slot | null {
  const id = hit ?? doc.rootId;
  if (!doc.nodes[id]) return null;
  if (isContainer(id)) return { parentId: id, index: indexIn(id) };
  return after(doc, id);
}

/** Enter, or a click, on a library item: into the selected container at the end, after a selected leaf, onto the page otherwise. */
export function slotForSelection(doc: Doc, selected: string, isContainer: (id: string) => boolean): Slot {
  if (!doc.nodes[selected]) return end(doc, doc.rootId);
  if (isContainer(selected)) return end(doc, selected);
  return after(doc, selected) ?? end(doc, doc.rootId);
}
```

`library-adds.ts` mirrors `layer-moves.ts` on purpose: a tree drop reuses the tree's own `placementAt` (a tile over a row reads exactly as a dragged layer does, and the row wears the same line or box, because `LayersPanel` takes an `insertion` from outside); a canvas drop takes the node under the pointer from the same `hitTest` the selection uses, and for a container the index the pointer has passed along its layout axis (`indexAlong`: how many children's middles it is past; flex row $\rightarrow$ x, else y); Enter or a click adds into a selected container at the end, *after* a selected leaf, onto the page otherwise. And, like the tree, it refuses only geometry: into a Button is a slot, and the replica refuses it.

FROM `apps/web/src/Canvas.tsx`

```
// The library's one add (E10.5): the id is minted HERE, random and never reused (SPEC §2.4); the slot is the
// library's (into or after the selection, or where a drag let go). The new node is selected, if the replica took it.
const addAt = (component: Component, slot: Slot): void => {
  const nodeId = crypto.randomUUID();
  if (submit(addOpAt(slot, component, nodeId))) setSelected(nodeId);
};
```

The id is minted *here*, in `Canvas`, with `crypto.randomUUID()`, random and never reused (SPEC §2.4: a reused id would let a late op for a removed node land on a new one), and the new node is selected only if the replica took the op. A refused add (a viewer, or the room later) would otherwise leave `wanted` pointing at a node that never existed, and the live region would say "removed by someone else"; the effect in `Canvas` that watches refusals for an `add_node` of the wanted id sends the selection back to the page instead.

FROM `apps/web/src/LibraryPanel.tsx`

```
const onPointerUp = (): void => {
  const p = press.current;
  if (!p) return;
  const component = componentOf(p.name);
  // A click (never travelled) adds where Enter would; a drag adds where it was let go, if that is a place.
  if (component && !p.moved) addToSelection(component);
  else if (component && over.current?.slot) add(component, over.current.slot);
  endDrag();
};
```

FROM `apps/web/src/Surface.tsx`

```
export function slotUnder(view: HTMLElement, point: Point, doc: Doc, isContainer: (id: string) => boolean): Slot | null {
  return slotOnCanvas(doc, hitTest(boxesOf(view), point), isContainer, (id) => {
    const wrapper = wrapperIn(view, id);
    const own = wrapper ? ownOf(wrapper) : null;
    if (!wrapper || !own) return 0;
    const axis = axisOf(own);
    return indexAlong(childRects(wrapper).map((r) => (axis === "x" ? { start: r.left, end: r.right } : { start: r.top, end: r.bottom })), axis === "x" ? point.x : point.y);
  });
}
```

The one exported function of `Surface` that is not a component: the library, holding a captured pointer, needs to know what slot the canvas would give a drop at a screen point, and `slotUnder` is the one place that reads the canvas's boxes for it, with the same measuring helpers the outlines use. The library then tells `Canvas` what it is carrying and over what (`onCarry`), and `Canvas` hands the tree an `insertion` or the canvas a `Slot` to draw, so the indicator is drawn by whichever pane owns the boxes.

FROM `apps/web/src/LibraryPanel.tsx`

```
<li
  key={component.name}
  role="option"
  aria-selected={component.name === active}
  aria-disabled={disabled ? true : undefined}
  tabIndex={component.name === tabStop ? 0 : -1}
  data-name={component.name}
  data-dragging={carrying === component.name ? "" : undefined}
  className="library-item"
  onPointerDown={event => { onPointerDown(event, component.name); }}
>
```

The APG *listbox*: `role=listbox`, `role=option` with `aria-selected`, roving `tabindex` as in the tree, a search box above that narrows the list (ArrowDown in it moves to the tiles: a scope of its own in the registry). Empty containers get 24 px and a dashed edge on the canvas, or an empty Stack could never be dropped into. The *ponytail* s are the honest ceilings: a two-column grid whose arrows walk in reading order (six components); a `src` prop gets an inline SVG so no tile makes a request for a name that is not an address (a failed request is a console error, and the e2e fixture fails on any).

TYPESCRIPT + REACT (THIS REPO)	JAVA (SWING, JAVAFX)	PYTHON (QT, TK)
three intents → one <code>Slot</code> → one <code>add_node</code>	a palette's <code>ActionListener</code> and a drop handler each mutate the model	the same, twice
<code>crypto.randomUUID()</code> minted once in the caller	<code>UUID.randomUUID()</code>	<code>uuid.uuid4()</code>
<code>role=listbox</code> / <code>option</code> , roving <code>tabindex</code>	<code>JList</code>	<code>QListWidget</code>
the tile is the real component, <code>inert</code> and <code>aria-hidden</code>	a <code>ListCellRenderer</code> painting a stamp	an item delegate

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

A tab beside Layers would have made the drop onto a row impossible.

The obvious pane layout was Layers and Library as two tabs of the left pane. A drag from a tile onto a *row* needs both on the screen at once, so the library sits under the layers. A layout decision forced by a gesture; worth naming because it is the kind of thing a mock-up hides.

7 · Presence in world coordinates, and a cursor for a peer that has no pointer

Epic 2 sent a cursor as a fraction of the canvas element, the right answer when every window drew the whole page in the same box. With zoom and pan per window a fraction of one canvas is a different spot on the sheet from a fraction of another; only a *world point* names the same component in both.

FROM `packages/contracts/src/index.ts`

```
const WORLD_LIMIT = 1_000_000;
const Cursor = z.strictObject({ x: z.number().min(-WORLD_LIMIT).max(WORLD_LIMIT), y: z.number().min(-WORLD_LIMIT).max(WORLD_LIMIT) });
const PresenceState = { cursor: Cursor.nullable(), selection: NodeId.nullable() };
```

FROM `apps/web/src/Surface.tsx`

```
const box = event.currentTarget.getBoundingClientRect();
// In WORLD coordinates, not screen pixels or a fraction of the canvas: the other window has its own size, zoom
// and pan, and this way the pointer lands on the same component there. Hundredths: a sub-pixel is noise on the wire.
const at = toWorld(viewport, { x: event.clientX - box.left, y: event.clientY - box.top });
onPoint({ x: hundredths(at.x), y: hundredths(at.y) });
setHovered(hitTest(boxesOf(event.currentTarget), { x: event.clientX, y: event.clientY }));
};
```

Out: `toWorld` of the pointer's position inside the canvas, rounded to hundredths (a sub-pixel is noise on the wire). The contract bounds the value at $\pm 1e6$ so a wild number never rides the wire, and takes any point of the infinite sheet: left of or above the frame is a place too. In: the mark is drawn *inside* `.world` at its world point, so this window's own transform places it on the same component, and it divides by `--zoom` so the arrow and the tag keep their screen size. Nothing is converted on the receiving side; the cursors unit test says it in arithmetic (two viewports, one world point, the same button), and the e2e spec says it with two browsers at different zooms and a tip within 4 px of the other's Button centre.

FROM `apps/web/src/app.css`

```
/* A cursor sits in .world at its world point and divides by --zoom like the labels, so the arrow and the tag keep their
   screen size at every zoom; the tip of the arrow is the element's corner, the point itself. Still for 5 s: it fades. */
.presence-cursor { position: absolute; pointer-events: none; transform-origin: 0 0; transform: scale(calc(1 / var(--zoom)));
transition: opacity 600ms ease; }
.presence-cursor[data-idle] { opacity: 0; }
/* A white edge on the glyph: the fill reads on the light sheet, the edge on the dark one. */
```

FROM `apps/web/src/cursors.ts`

```
export function trackCursors(previous: ReadonlyMap<string, Tracked>, others: readonly Presence[], now: number): Map<string,
Tracked> {
  const next = new Map<string, Tracked>();
  for (const p of others) {
    if (!p.cursor) continue;
    const was = previous.get(p.peerId);
    next.set(p.peerId, was && was.at.x === p.cursor.x && was.at.y === p.cursor.y ? was : { at: { x: p.cursor.x, y: p.cursor.y
}, movedAt: now });
  }
  return next;
}

export const isIdle = (tracked: Tracked, now: number): boolean => now - tracked.movedAt >= IDLE_AFTER_MS;
```

FROM `apps/web/src/Canvas.tsx`

```
// The others' cursors with the time each last moved, so a still one fades (E10.6). A ref, renewed on each
// render from what the peer says now: the moved-at times are the only thing remembered between renders.
const tracked = useRef<Map<string, Tracked>>(new Map());
const now = Date.now();
tracked.current = trackCursors(tracked.current, peer?.others ?? [], now);
// A cursor fades by the clock, not by an event: while any is shown, a tick a second re-reads the clock.
const [, setTick] = useState(0);
const anyCursor = tracked.current.size > 0;
useEffect(() => {
  if (!anyCursor) return;
  const timer = setInterval(() => { setTick((n) => n + 1); }, 1000);
  return () => { clearInterval(timer); };
}, [anyCursor]);
```

Idle is *by the clock, not by an event*. A presence refresh repeats the same spot every 2 s, and that is not a move; `trackCursors` keeps the old `movedAt` when the point is unchanged. Nothing arrives when a pointer rests, so no event could fade it; while any cursor is shown a one-second tick re-reads `Date.now()` and the mark's `data-idle` attribute drives a CSS opacity transition. Note the shape in `Canvas`: the tracked map is a `useRef` renewed on every render from what the peer says now, the moved-at times being the only thing remembered between renders, and the tick is a `useState` whose value nobody reads: its only job is to cause a render.

FROM `packages/peer-client/src/peer.ts`

```
/**
 * Every op the room has ORDERED, ours included, with the actor the room stamped on it, told after it is
 * applied. A window onto who did what (E10.6: the AI's cursor sits on the node its last op touched); not
 * a place to edit from, and nothing here depends on it being listened to.
 */
onOp?: (message: SequencedOp) => void;
```

FROM `packages/peer-client/src/peer.ts`

```
const effects = replica.receive(message);
for (const { opId, outcome } of effects.settled) settle(opId, outcome); // BEFORE a fatal end: these ops have left
`pending`, so abandon() could not report them
```

```

    if (effects.fatal) { finish(effects.fatal); return; }
    if (message.type === "op") onOp?.(message);

```

FROM `apps/web/src/cursors.ts`

```

export function aiAnchors(previous: ReadonlyMap<string, string>, message: SequencedOp): ReadonlyMap<string, string> {
  if (message.actor.kind !== "agent" || message.op.type === "remove_node") return previous;
  return new Map(previous).set(message.actor.id, message.op.nodeId);
}

/** Where an AI's cursor sits now: its anchor while that node exists, else the page (an AI that has just arrived, or whose
node went). */
export const anchorFor = (anchors: ReadonlyMap<string, string>, actorId: string, doc: Doc): string => {
  const at = anchors.get(actorId);
  return at !== undefined && doc.nodes[at] ? at : ROOT_ID;
};

```

The AI has no pointer, so presence cannot say where it is. But the room stamps every ordered op with its actor (keystone 3: stamped by the room from the verified session, never sent by the peer), and the peer already applies those ops; `onOp` tells a listener after each apply. It is a *window onto who did what, not a write path*: the listener is told after the fact, returns `void`, and nothing in the peer depends on anyone listening. `aiAnchors` keeps one anchor per agent actor id from its add, move and set_prop ops (two runs are two anchors) and hands back the *same* map for anyone else's op, so React sees no change and renders nothing for a person's edits. `Surface`'s measuring pass, the one that finds the outlines' boxes, also finds where the anchored node is; the mark is a spark set inside the node's corner, where a pointer would not be; before the first op, or when the node is gone, it sits on the page. The `ponytail`: a removal leaves the anchor where it was, because the op does not carry the parent and the listener hears it after apply; a `parentId` on the broadcast is the upgrade.

FROM `apps/web/src/Canvas.tsx`

```

// The others on the sheet (E10.6): a person where their pointer is, in world coordinates; the AI on the node
// its last op touched (the page until it has touched one). Someone whose pointer has left the canvas has no mark.
const marks = peer.others.flatMap((p): CursorMark[] => {
  const colour = colourOf(p.peerId);
  if (p.actor.kind === "agent") return [{ peerId: p.peerId, kind: p.actor.kind, label: tagOf(p), colour, idle: false, at: {
nodeId: anchorFor(anchors, p.actor.id, doc) } }];
  const seen = tracked.current.get(p.peerId);
  return seen ? [{ peerId: p.peerId, kind: p.actor.kind, label: tagOf(p), colour, idle: isIdle(seen, now), at: seen.at } ] :
[];
});

```

The bar is the *record* of who is here; the cursors are decoration. An avatar per connection (initials on the person's colour, white text: the palette is checked for that) is a button that makes their selection ours and pans the canvas to centre it (`centreOn`, one more pure function of the viewport), disabled while they have none; one polite sentence says the list when it changes. The marks live in the editor layer, which is `aria-hidden`: decoration must not talk, and the e2e spec asserts the cursor has an `aria-hidden` ancestor.

TYPESCRIPT + REACT (THIS REPO)	JAVA (SWING, JAVAFX)	PYTHON (QT, TK)
a cursor is a world point, bounded, to the hundredth	your own protocol; the same lesson applies	the same
<code>onOp</code> : told after apply, returns void	a read-only <code>ModelListener</code>	a signal with no slot that writes
idle by a 1 s tick reading <code>Date.now()</code>	a <code>javax.swing.Timer</code>	a <code>QTimer</code>
the same <code>Map</code> back = no render	<code>equals</code> before <code>firePropertyChange</code>	compare before <code>emit</code>

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

Mapping the fractional cursor through the frame's box broke the moment either side panned.

The cheap fix for E10.2's zoom was to keep the fraction and divide by the frame's box on each side. It held while both windows showed the whole frame and broke as soon as one panned: a fraction of a *viewport* is not a fraction of the *frame*. The wire changed instead, to the one coordinate system that is the same everywhere, and the e2e spec zooms and pans both sides before it measures.

8 · One shortcut registry, a native dialog, and a preview that is the device's real width

FROM `apps/web/src/shortcuts.ts`

```
export const SHORTCUTS = [
  { scope: "global", action: "help", keys: ["?"], does: "Show this sheet" },
  { scope: "global", action: "preview", keys: ["p", "P"], does: "Show or hide the running page beside the canvas" },

  { scope: "canvas", action: "next", keys: ["ArrowRight", "ArrowDown"], does: "Select the next element" },
  { scope: "canvas", action: "previous", keys: ["ArrowLeft", "ArrowUp"], does: "Select the previous element" },
  { scope: "canvas", action: "in", keys: ["Enter"], shift: false, does: "Go into the element, to its first child" },
  { scope: "canvas", action: "out", keys: ["Enter"], shift: true, does: "Go to the element's parent" },
  { scope: "canvas", action: "page", keys: ["Escape"], does: "Select the page" },
  { scope: "canvas", action: "zoom-in", keys: ["+", "="], does: "Zoom in" },
  { scope: "canvas", action: "zoom-out", keys: ["-", "_"], does: "Zoom out" },
  { scope: "canvas", action: "fit", keys: ["0"], does: "Fit the page in the view" },
  { scope: "canvas", action: "pan", keys: [" "], does: "Hold, and drag to pan" },
```

FROM `apps/web/src/shortcuts.ts`

```
type Entry = (typeof SHORTCUTS)[number];
type Actions = { [S in Scope]: Extract<Entry, { scope: S }>["action"] };
/** The actions one scope declares, as a type: a handler's switch is checked against the table. */
export type ActionOf<S extends Scope> = Actions[S];
/** What `actionFor` reads of a keydown: a React or a native KeyboardEvent, or a plain object in a test. */
export type KeyLike = Pick<KeyboardEvent, "key" | "altKey" | "shiftKey" | "ctrlKey" | "metaKey">;

const binds = (s: Shortcut, event: KeyLike): boolean => s.keys.includes(event.key) && (s.alt ?? false) === event.altKey &&
(s.shift === undefined || s.shift === event.shiftKey);

/** The action a keydown means in `scope`, or null: not a shortcut here, or a browser chord (Ctrl, Cmd). */
export function actionFor<S extends Scope>(scope: S, event: KeyLike): ActionOf<S> | null {
  if (event.ctrlKey || event.metaKey) return null;
  // The scope was checked, so the action is one of that scope's: the type only knows it once told.
  const found = SHORTCUTS.find((s) => s.scope === scope && binds(s, event));
  return (found?.action ?? null) as ActionOf<S> | null;
}
```

Every key the editor answers is a row: a scope (where it works: the focused widget, or `global` for everywhere the person is not typing), an action, the `KeyboardEvent.key` values, Alt and Shift, and a sentence. The table drives two things. The *handlers*: `Surface`, `LayersPanel`, `LibraryPanel`, `Inspector`, the splitter and `Canvas`'s global listener call `actionFor(scope, event)` and switch on the action, never on `event.key`. The *sheet*: `ShortcutSheet` lists the same rows by scope. So a key the sheet promises is a key that works. The TypeScript is worth reading slowly: as `const` satisfies `readonly Shortcut[]` keeps the literal types (each `action` is a string literal, not `string`) while checking the shape; `ActionOf<S>` is then *derived from the table*, so a handler's `switch` is checked against the rows of its scope, and adding a row is the whole of adding a key (drill 1). Ctrl and Cmd are never bound: those chords are the browser's, and `actionFor` answers null for them. `shortcuts.test` fails on two rows one keydown could mean, in one scope or global against any.

FROM `apps/web/src/shortcuts.ts`

```
export const typingIn = (target: { tagName: string; isContentEditable: boolean }): boolean => ["INPUT", "TEXTAREA", "SELECT"].includes(target.tagName) || target.isContentEditable;
```

FROM `apps/web/src/Canvas.tsx`

```
// The global shortcuts (shortcuts.ts, scope "global"): anywhere in the editor, unless the person is typing
// or a modal is up. A widget's own handler runs first and, finding no action of its own, lets the key bubble here.
useEffect(() => {
  const onKeyDown = (event: KeyboardEvent): void => {
    if (sheetOpen || shareOpen || (event.target instanceof HTMLElement && typingIn(event.target))) return;
    const action = actionFor("global", event);
    if (action === "help") setSheetOpen(true);
    else if (action === "preview") setPreviewOpen((open) => !open);
    else return;
    event.preventDefault();
  };
  window.addEventListener("keydown", onKeyDown);
  return () => { window.removeEventListener("keydown", onKeyDown); };
}, [sheetOpen, shareOpen]);
```

The global keys (`?` , `P`) are a `window` listener added in an effect, and they are ignored while the person is typing (`typingIn` : a `?` typed into the library's search box is a `?`) or while a modal is up. Order matters: a widget's own handler runs first and, finding no action of its own, lets the key bubble to the window; that is why `Surface` 's handler returns early on `null` instead of calling `preventDefault` for everything.

FROM `apps/web/src/ShortcutSheet.tsx`

```
export function ShortcutSheet({ open, onClose }: { open: boolean; onClose: () => void }) {
  const dialog = useRef<HTMLDialogElement>(null);
  const titleId = useId();
  useEffect(() => {
    const el = dialog.current;
    if (!el) return;
    if (open && !el.open) el.showModal();
    else if (!open && el.open) el.close();
  }, [open]);
```

The sheet is a native `<dialog>` shown with `showModal()` : the browser traps focus inside it, makes the rest of the page `inert` , closes it on Escape and gives focus back to where it was, all without a line of trap code, and axe likes it. The React shape is the one to copy: `open` is the caller's state; an effect tells the element to match it; the element's own `close` event (Escape, the button) tells the caller. A click on the dialog element itself is a click on the backdrop, because the body fills the dialog. The Share dialog (section 9) is the same shape with a form in it.

FROM `apps/web/src/preview-split.ts`

```
export function frameScale(pane: Size, device: Size, padding = 16): number {
  const fitted = Math.min((pane.width - 2 * padding) / device.width, (pane.height - 2 * padding) / device.height);
  return Number.isFinite(fitted) ? Math.min(1, Math.max(0.05, fitted)) : 1;
}
```

FROM `apps/web/src/Preview.tsx`

```
    /* The fit box takes the SCALED size in the layout; the frame inside is the device's real size, scaled down by CSS
    alone. */
    <div className="device-fit" style={{ width: device.width * scale, height: device.height * scale }}>
      <div className="device-frame" data-device={device.name.toLowerCase()} data-scale={scale.toFixed(3)} style={{ width:
device.width, height: device.height, transform: `scale(${String(scale)})` }}>
        /* keyed on the URL: a new address is a new page, an iframe whose src changes keeps nothing worth keeping */
        {url && <iframe key={url} title="Preview of this page" src={url} sandbox="allow-scripts" />}
      </div>
    </div>
```

The preview was a strip under the canvas at one width. Now it is a resizable split beside the canvas, and the page runs in a *device frame* that is the device's *real* CSS size (390, 768 or 1280 wide), scaled down by one `transform` to fit the pane and never above life size. So the page inside measures itself at 390 px and its own media queries answer; scaling the iframe by a `width` attribute was rejected because the page would then see the scaled width. The bezel is a box-shadow, never a border, so it takes nothing from the width (a bordered 390 would leave the page 374). `useMeasured` is a `ResizeObserver` in a `useLayoutEffect` that sets state only when the size changed, the same no-render-for-no-change shape as the outlines. Every isolation rule from E4.3 is unchanged and `preview.spec` still asserts them: `sandbox="allow-scripts"` without `allow-same-origin` , the URL asked for each second and never kept, loopback or own origin only, the container's token in the path; and the preview mounts only while open, because an open preview holds a container.

FROM `apps/web/src/Preview.tsx`

```
<div
  role="separator"
  aria-orientation="vertical"
  aria-label="Preview width"
  aria-valuenow={width}
  aria-valuemin=MIN_PREVIEW
  aria-valuemax={maxPreviewWidth(size.width)}
  aria-valuetext={` ${String(width)} pixels`}
  tabIndex={0}
  className="splitter"
  onPointerDown={onPointerDown}
  onPointerMove={onPointerMove}
  onPointerUp={() => { drag.current = null; }}
  onPointerCancel={() => { drag.current = null; }}
```

```
onKeyDown={onKeyDown}
/>
```

The divider is the APG *window splitter*: `role=separator` with `aria-orientation`, `aria-valuenow/min/max` and a `valuetext`, focusable, dragged by pointer with capture or moved 24 px at a time with Left and Right through the registry's `splitter` scope. Its `ponytail` s: the width and the device are a session's choice, not remembered; the narrow layout stacks the two with no divider.

TYPESCRIPT + REACT (THIS REPO)	JAVA (SWING, JAVAFX)	PYTHON (QT, TK)
one table; <code>actionFor(scope, event)</code> ; <code>ActionOf<S></code> derived from it	<code>InputMap / ActionMap</code> per component; <code>KeyStroke.getKeyStroke("alt UP")</code>	<code>QAction.setShortcut</code> ; <code>QShortcut</code> with a context
a test that no two rows collide	none; the last <code>put</code> wins silently	Qt warns of an ambiguous shortcut at runtime
<code><dialog>.showModal()</code> : focus trap, inert page, Escape, focus return, free	<code>JDialog</code> with <code>setModal(true)</code>	<code>QDialog.exec()</code>
a device frame of real CSS width, scaled by one transform	n/a	n/a

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

P as Shift+P, and a hand-rolled focus trap, both rejected.

Two habits from other toolkits were proposed and refused. Binding the preview to Shift+P "so it cannot fire while typing": a bare letter outside fields is what Figma does, and `typingIn` already keeps it out of the search box; the chord would have been a second answer to a solved problem. And a focus trap of our own for the sheet: forty lines that a native modal dialog does better, with the page made `inert` for free, which no hand-rolled trap can do.

9 · Screens for what the api already does: a role as a hint, and a list that is never guessed

E10.8 adds no ability. The api could already list members and change roles (E8.2), share and revoke (E8.3); it lacked two *read* routes and every screen. The section is about what a screen may decide, which is nothing, and about when *not* to be optimistic.

FROM `apps/api/src/app.ts`

```
// E10.8: who is in the org, with their roles. Every member: the same people a member already meets in every document
// here, and the screen that lets an owner change roles is the one a viewer reads. Only THIS org's rows (the query says so).
org.get("/members", need("viewer"), async (c) => {
  const page = await c.var.scope.listMembers(pageQuery(c));
  return page ? c.json(page) : badCursor(c);
});
```

FROM `apps/api/src/app.ts`

```
document.get("/shares", need("owner"), async (c) => {
  const page = await db.forOrg(c.var.doc.orgId).listShares(c.var.doc.id, pageQuery(c));
  return page ? c.json(page) : badCursor(c);
});
```

Both routes are rows in the E8.2 route-role table, which `app.test` checks: a route without a `need` fails the test. Members are readable by a viewer (every member meets these people in every document already, and the screen an owner changes roles on is the one a viewer reads); shares are owners-only, as sharing itself is (the list names people outside the org, and an editor has no business with it). Both page like every other list, over a join handed to `page()` as the table, with the org and the document in the WHERE: a list never reaches past the tenant the middleware already proved.

FROM `packages/contracts/src/index.ts`

```
export const SessionResponse = z.object({ wsUrl: z.url(), token: z.string().min(1), expiresAt: Timestamp, role:
Role.optional() });
```

FROM apps/web/src/Canvas.tsx

```
{role === "owner" && <button type="button" className="primary" aria-haspopup="dialog" onClick={() => {
  setShareOpen(true); }}>Share</button>}
```

The editor shows Share to an owner only, and it must know that without asking a route that would answer 403 (the e2e fixture fails on any console error, and a viewer's page must stay clean). So the session response gains an optional `role`: what the document middleware read from the same row that proved access, *beside* the token and never in it. E1.5 stands: the token says who; the routes and the room read the row for themselves on every request and every op. `usePeer` keeps the latest session's role (a reconnect mints a new session and passes here too, so the hint follows the api's latest word), and `Canvas` shows Share and mounts the dialog only for `"owner"`. A hint for the screen, never authority: a role that changes under an open page is met as a sentence from the api at the next action, and by the room sooner, on its next Redis hint or 30 s sweep (E8.3). Rejected: probing `GET /shares` on open (a 403 in the console for every non-owner) and putting the role in the token (a token is minted once and read many times; a role changes).

FROM apps/web/src/members.ts

```
const MEMBER_REFUSALS: Record<MemberRefusal, string> = {
  no_user: "Nobody has signed up with that email yet. Ask them to sign up first, then add them.",
  // The 409 last_owner, as a sentence: the org would be left with nobody to run it (F24).
  last_owner: "An organisation must keep at least one owner. Make someone else an owner before changing this role.",
  forbidden: "Only an owner of this organisation can add members or change roles.",
  gone: "This organisation cannot be found, or you are no longer a member of it.",
};
```

FROM apps/web/src/members.ts

```
export function withMember(members: readonly Member[], member: Member): Member[] {
  return members.some((m) => m.userId === member.userId) ? members.map((m) => (m.userId === member.userId ? member : m)) :
  [...members, member];
}
export const withoutMember = (members: readonly Member[], userId: string): Member[] => members.filter((m) => m.userId !==
userId);
```

FROM apps/web/src/OrgView.tsx

```
const apply = async (at: string, email: string, role: Role): Promise<boolean> => {
  let result;
  try {
    result = await setMemberRole(orgId, email, role);
  } catch {
    setProblem({ at, text: "The change could not be saved. Try again." });
    return false;
  }
  if (typeof result === "string") {
    setProblem({ at, text: memberRefusalWords(result) });
    if (result === "forbidden" || result === "gone") { setChanged([]); reload(); } // the world moved under this page: read
it again
    return false;
  }
  setProblem(undefined);
  setChanged((before) => withMember(before, result));
  setSaid(`${result.email} is now ${result.role}.`);
  return true;
};
```

Here is the contrast with the document. The replica is *optimistic*: an edit is shown before the room answers, because the ops were designed for it (small, validated locally by the same rules, rebuilt from a journal on refusal). A membership change is *not*: `withMember` applies the api's own answer, never the request; the 409 `last_owner`, the 404 `no_user` and a 403 are sentences from a `Record` over the api's whole vocabulary (a name added to the api's answers stops this file compiling until it has words); and a 403 or 404 makes the page drop its local changes and read the list again, because the world moved under it. The caller's own role is read from the list they were given (`roleOf`): the member list already carries it, so no route need say it twice. Every name and email is a React text node: React escapes text children, so a name of `<img onerror=...>` is letters on the screen.

FROM apps/web/src/ShareDialog.tsx

```
const revoke = async (member: Member): Promise<void> => {
  let result;
```

```

    try {
      result = await revokeShare(documentId, member.userId);
    } catch {
      setProblem("The share could not be revoked. Try again.");
      return;
    }
    setConfirming(null);
    if (result === "forbidden") { setProblem(shareRefusalWords("forbidden")); return; }
    setProblem(undefined);
    setShares((before) => withoutMember(before ?? [], member.userId));
    setSaid(`The share of ${member.email} was revoked. Their open session is closing.`);
    email.current?.focus(); // the row is gone: focus must not fall out of the dialog
  };
  const keep = (member: Member): void => {
    setConfirming(null);
    // Back to the button that asked, once it is drawn again.
    setTimeout(() => { dialog.current?.querySelector<HTMLButtonElement>(`[data-revoke="${member.userId}"]`)?.focus(); }, 0);
  };

```

The Share dialog is the sheet's shape with a form: a native `<dialog>`, the list read afresh every time it opens, add at viewer or editor, change a role, and Revoke that asks *in place*. Focus management is the craft here: the confirmation appears where the person was and `autoFocus` lands on the answer they came for; Keep gives focus back to the button that asked, once it is drawn again (a `setTimeout(0)`, because the button does not exist until the next render); a revoke removes the row, so focus goes to the email field rather than falling out of the dialog. The global shortcuts stay quiet while it is up. The `ponytail`: the first page of shares (50); a 51st is not listed; a "show more" as the org pages have is the upgrade.

FROM `apps/web/src/links.ts`

```

export function hrefTo(page: Record<string, string>, search: string = location.search): string {
  const user = new URLSearchParams(search).get("user");
  const params = new URLSearchParams(page);
  if (user !== null) params.set("user", user);
  return `/${params.toString()}`;
}

```

Home lists your organisations, and an org's pages (members, audit trail, AI usage) share one nav with `aria-current`. `hrefTo` is the address bar as the router (`?doc=`, `?org=` ...) and carries the development `?user=` along, so two windows stay two people from one page to the next; with a real sign-in there is none and the link is just the page.

TYPESCRIPT + REACT (THIS REPO)	JAVA (SWING, JAVAFX)	PYTHON (QT, TK)
the members list applies the api's answer; a 403 re-reads	a <code>TableModel</code> updated from the response	the same
<code>role</code> beside the token in the session, read from the row	a <code>Principal</code> 's roles in the session; re-checked per request by Spring Security	Django's <code>request.user</code> , re-checked per view
<code>Record<Refusal, string></code> : a new refusal name is a compile error	an <code>EnumMap</code> with a <code>switch</code> that is not exhaustive	a <code>dict</code> and a <code>KeyError</code> in production
React text nodes: escaped by construction	Swing's <code>JLabel</code> renders <code><html></code> if the text starts with it	Qt rich text if the string looks like HTML

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

Probing GET /shares on open would have put a 403 in every viewer's console.

The first idea for "is this person an owner" was to ask `/shares` when the editor opens and show Share if it answers. Every viewer and editor would have opened the document to a 403 in the console, which the e2e fixture treats as a failure, and which a support engineer would read as a bug. The session already knew the role from the row it proved access by; saying it beside the token cost one optional field and no authority.

10 · Trade-offs and comparisons: where this editor sits

Retained widgets versus a view of the state. Swing, JavaFX, Qt and Tk are *retained-mode*: a widget tree is built once and mutated by events, and the toolkit repaints the damage. React is *declarative over a retained DOM*: your code describes the whole screen from state on every change and React mutates the DOM to match. The desktop model is faster per event and puts the truth in the widgets, where it drifts (the classic Swing bug is a `TreeModel` that no longer matches the data). The React model puts the truth in one place (here the replica) and makes every pane a projection; the cost is that render must be cheap and pure, which is why the pure modules of this epic are small and the components thin. JavaFX's property bindings and Qt's model/view are each half-way to this: the view updates from a model, but the model is yours to keep in step.

Optimistic versus confirmed. The document is optimistic because the replica can afford it: the same `validate()` runs locally, a refusal is a rebuild from `confirmed` plus the remaining pending ops, and the invariant is tested as a property. The members list is confirmed-only because its truth is rows behind locks and side effects, and a guessed row would lie. The rule of thumb: be optimistic where you can run the server's rule locally and recover from a refusal by recomputation; otherwise show the answer.

One transform versus re-layout. A canvas that re-lays its content out at every zoom (setting widths and font sizes) is how a browser's own page zoom works and is right for text. A design canvas wants the frame to be a picture that scales: one `transform` on one layer, composited, with the editor's own strokes divided back to screen size. Figma does the same on a WebGL canvas; here the "picture" is live DOM, so the components stay real and inspectable, and the price is that measuring must happen after render.

HTML drag and drop versus pointer events. HTML DnD gives you cross-window drags, a ghost image and the OS's drop semantics, at the cost of a second event vocabulary, a ghost to fight, and `dragover` on its own clock. Pointer events with capture give one vocabulary for the canvas, the tree and the library, an indicator that follows the exact pointer, and Escape to cancel; they cannot drag out of the window, which nothing here needs.

Where Figma differs. Figma's canvas is freeform vectors on a WebGL surface with its own layout engine, and multiplayer is a CRDT-like property map with last-writer-wins per property. Noon's canvas is a *tree of real components* laid out by the design system itself (SPEC §5: no freeform drawing, no absolute positioning, so the design can become code), and multiplayer is a server-ordered op log (the note on multiplayer approaches has the comparison). The Figma *feel* (a dotted infinite sheet, zoom at the pointer, layers left and inspector right, one accent, a `?` sheet, cursors with name tags) is borrowed; the model is not.

THIS REPO	JAVA DESKTOP	PYTHON DESKTOP
React: describe the screen from state; diff the DOM	Swing/JavaFX: mutate widgets; repaint damage	Qt/Tk: mutate widgets; repaint damage
one replica, every pane a projection	a model per widget you keep in step	model/view with the model yours to keep
optimistic where the rule runs locally, confirmed otherwise	usually confirmed (a blocking call on the EDT is the anti-pattern)	usually confirmed
a11y by ARIA roles, checked by axe in a test	the L&F's accessibility bridge	Qt's accessibility plugin
theming by custom properties in the cascade	Look-and-Feel objects	palettes and stylesheets

11 · Testing a screen: pure halves as arithmetic, a WebSocket that counts frames, axe in both themes

The split from every earlier lesson holds, and the frontend makes it sharper: everything that can be a pure function is (viewport maths, hit-testing, drop-to-op, prop-to-control, slot-to-op, cursor tracking, the registry), tested as arithmetic in milliseconds; everything that needs a browser is an end-to-end spec against the real stack, written and unrun. Five shapes are new.

FROM `apps/web/src/theme.test.ts`

```
// [text, on surface, minimum]: 4.5:1 for text, 3:1 for borders, focus rings and the canvas dots' frame.
const PAIRS: [string, string, number][] = [
  ["fg", "surface", 4.5], ["fg", "bg", 4.5], ["fg", "canvas", 4.5], ["fg", "surface-2", 4.5], ["fg", "accent-soft", 4.5],
  ["fg-muted", "surface", 4.5], ["fg-muted", "bg", 4.5], ["fg-muted", "canvas", 4.5],
  ["accent", "surface", 4.5], ["accent", "bg", 4.5], ["accent", "canvas", 4.5], ["on-accent", "accent", 4.5],
  ["danger-fg", "danger-bg", 4.5], ["warning-fg", "warning-bg", 4.5], ["info-fg", "info-bg", 4.5],
  ["border-strong", "surface", 3], ["border-strong", "canvas", 3], ["accent", "surface-2", 3],
```

```
[ "danger-border", "danger-bg", 3], [ "warning-border", "warning-bg", 3], [ "info-border", "info-bg", 3],
];
```

Accessibility as arithmetic. The contrast of every text/surface pair the UI uses, recomputed from `tokens.css` itself in both themes. A colour token is data, and data can be tested; the same idea checked the presence palette in E2.6.

FROM `apps/web/src/controls.test.ts`

```
test("every prop of every component in the committed manifest gets a control of the kind its type asks for", () => {
  const KIND_TO_CONTROL = { string: ["text"], number: ["number"], boolean: ["toggle"], enum: ["segmented", "select"] };
  let seen = 0;
  for (const component of manifest.components) {
    for (const each of component.props) {
      const control = controlFor(each);
      expect(KIND_TO_CONTROL[each.type.kind], `${component.name}.${each.name}`).toContain(control.kind);
      expect(control.prop).toBe(each);
      seen++;
    }
  }
  expect(seen).toBeGreaterThan(0);
});
```

The committed manifest as a fixture. The inspector's promise is "every prop has a control"; the test walks the real manifest rather than a hand-made one, so a regenerated manifest that breaks the promise breaks the test.

FROM `apps/web/src/cursors.test.ts`

```
// unit:cursor-world-coords (E10.6)
test("a pointer is sent in WORLD coordinates: two viewports at different zooms and offsets name the same spot on the frame",
() => {
  const mine: Viewport = { x: 200, y: 80, zoom: 0.5 };
  const theirs: Viewport = { x: -300, y: 10, zoom: 2.4 };
  // I point at the frame's (240, 100): the middle of a button, say.
  const onMyScreen = toScreen(mine, { x: 240, y: 100 });
  const sent = toWorld(mine, onMyScreen);
  expect(sent.x).toBeCloseTo(240, 9);
  expect(sent.y).toBeCloseTo(100, 9);
  // Drawn on their screen at THEIR zoom, it lands where their button is drawn.
  const drawn = toScreen(theirs, sent);
  expect(drawn).toEqual(toScreen(theirs, { x: 240, y: 100 }));
  // A world point is any point of the infinite sheet: left of or above the frame is a place too, and the wire takes it.
  for (const cursor of [at(-40.5, 1200), at(0, 0), at(960, 600), at(1e5, -1e5)]) expect(ClientMessage.safeParse({ type:
"presence", cursor, selection: null }).success, JSON.stringify(cursor)).toBe(true);
  // ...but not a number that is not one, or one that could never be a spot on a page.
  for (const cursor of [{ x: Number.NaN, y: 0 }, { x: Number.POSITIVE_INFINITY, y: 0 }, { x: 1e7, y: 0 }, { x: 0, y: 0 }, { x:
"left", y: 0 }]) expect(ClientMessage.safeParse({ type: "presence", cursor, selection: null }).success,
JSON.stringify(cursor)).toBe(false);
});
```

Two viewports, one point. The claim "cursors agree across zoom" is a statement about `toWorld` and `toScreen`, and the unit test says it with two viewports and the contract's parser; the e2e spec says it again with two browsers and a tip within 4 px, which is the wiring.

FROM `e2e/layers-drag-and-keyboard-reorder.spec.ts`

```
/** Counts the move_node ops THIS browser sends: the proof that a drag is one op, not a remove and an add, nor one per pixel.
*/
async function countingMoves(page: Page): Promise<{ moves: () => number }> {
  let moves = 0;
  await page.routeWebSocket(/\documents\\/, (ws) => {
    const server = ws.connectToServer();
    ws.onMessage((message) => {
      const parsed: unknown = JSON.parse(String(message));
      if (typeof parsed === "object" && parsed !== null && "op" in parsed && typeof parsed.op === "object" && parsed.op !==
null && "type" in parsed.op && parsed.op.type === "move_node") moves++;
      server.send(message);
    });
    server.onMessage((message) => { ws.send(message); });
  });
  return { moves: () => moves };
}
```

A routed WebSocket that counts frames. "One op per move" is the chapter's thesis, and a spec that only looked at the resulting tree could not tell one `move_node` from a remove and an add, nor from one op per pixel. Playwright's `routeWebSocket` sits between the browser and the server, forwards every frame, and counts the ones whose op is a `move_node`; the spec asserts the count after each drag. `inspector-edits-propagate` does the same for `set_prop`: one per edit, none per keystroke.

FROM `e2e/fixtures.ts`

```
cleanPage: [
  async ({ page, allowedConsole }, use) => {
    const problems: string[] = [];
    page.on("console", (msg) => {
      if ((msg.type() === "error" || msg.type() === "warning") && !allowedConsole?.test(msg.text()))
        problems.push(`${msg.type()}: ${msg.text()}`);
    });
    page.on("pageerror", (err) => { if (!allowedConsole?.test(err.message)) problems.push(`pageerror: ${err.message}`); });

    await use(undefined);

    expect(problems, "browser console must stay clean").toEqual([]);
    const axe = await new AxeBuilder({ page }).analyze();
    expect(axe.violations.map((v) => `${v.id}: ${v.help}`), "no axe violations").toEqual([]);
  },
];
```

FROM `e2e/editor.ts`

```
export const axeClean = async (page: Page, theme: string): Promise<void> => {
  const axe = await new AxeBuilder({ page }).analyze();
  expect(axe.violations.map((v) => `${v.id}: ${v.help}`), `no axe violations in ${theme}`).toEqual([]);
};

/** Axe on the page as it is (the spec put it in light), then again after the bar's Dark theme. */
export const axeCleanLightAndDark = async (page: Page): Promise<void> => {
  await axeClean(page, "light");
  await button(page, "Dark theme").click();
  await expect(page.locator("html")).toHaveAttribute("data-theme", "dark");
  await axeClean(page, "dark");
};
```

Axe and a clean console, for free, in both themes. Every e2e test ends with two checks it did not write: the browser console stayed clean (a failed image request, a React warning, an unhandled rejection all fail the test) and the page has no axe violations. Because the fixture runs axe once in whatever theme the test left, the E10 specs also call `axeCleanLightAndDark`, and `shell-light-dark` takes screenshot baselines in both. After E10.6 those helpers and the editor's locators were copied from spec to spec; `noon-920` moved them into `e2e/editor.ts` because `jscpd` found six clones, and made `Preview` a plain function because `knip` found it exported and unused, rejecting an ignore entry that would have hidden the next one.

FROM `e2e/shell-light-dark.spec.ts`

```
// Light, as the OS says; screenshot and axe.
const toggle = page.getByRole("button", { name: "Dark theme", exact: true });
await expect(toggle).toHaveAttribute("aria-pressed", "false");
await expect(html(page)).toHaveAttribute("data-theme", "light");
await axeClean(page, "light");
await expect(page).toHaveScreenshot("shell-light.png");

// The toggle, by keyboard: dark, remembered across a reload.
await toggle.focus();
await page.keyboard.press("Space");
await expect(toggle).toHaveAttribute("aria-pressed", "true");
await expect(html(page)).toHaveAttribute("data-theme", "dark");
await axeClean(page, "dark");
await expect(page).toHaveScreenshot("shell-dark.png");
await page.reload();
await expect(page.getByRole("status")).toHaveText("live");
await expect(html(page)).toHaveAttribute("data-theme", "dark");
// ...and it now outranks the OS.
await page.emulateMedia({ colorScheme: "light" });
await expect(html(page)).toHaveAttribute("data-theme", "dark");
```

<code>expect(toWorld(v, toScreen(v, p))).toBeCloseTo(p)</code>	<code>assertEquals(p, t.inverseTransform(t.transform(p)), 1e-9)</code>	<code>pytest.approx</code>
<code>page.routeWebSocket(...)</code> counting frames	a proxy in the test; rare	the same
<code>new AxeBuilder({ page }).analyze()</code>	no equivalent in the JDK; manual audits	none
<code>toHaveScreenshot("shell-dark.png")</code>	manual	manual
<code>readFileSync("tokens.css")</code> as a fixture	a resource file read in the test	the same

CASE STUDY · FOUND BY REVIEW, BY A LEARNING TEST, OR BY A TEST THAT WAS WRITTEN TO REFUTE A CLAIM

A spec that pressed "Select X" buttons broke when the role moved, and the names did not.

Five earlier specs reached the layers as buttons; E10.3 made them treeitems. The specs changed one locator each (`getByRole("treeitem", { name })`) and nothing else, because the accessible *names* were unchanged. Locating by role and name, the way a screen reader does, is what let the UI change shape under the tests without the tests changing meaning; and it is also why every E10 spec asserts a sentence ("Button does not take children") rather than a class.

Honest status, repeated here because it matters: the unit tests named in the status block ran green on every commit. The eleven end-to-end specs of this epic and `members-shares.int` are written, reviewed and committed, and have not been run; the screenshot baselines must be regenerated when they are. Section 12 is what running them looks like.

12 · By hand: the theme, the zoom, a drag with the network tab open, two cursors, the sheet, Share

The exact commands and what each should show. The api is on host port 3000, the demo's web app on 5173; run a Vite of your own on 5199 so the demo is left alone. Two browser windows signed in as two people (`?user=` in development) are the whole harness for the collaboration steps.

0 · BRING THE STACK UP, AND AN EDITOR OF YOUR OWN

```
./init.sh # the stack; no new migrations in this epic
pnpm exec vite --config apps/web/vite.config.ts --port 5199
open 'http://localhost:5199/?user=ada@example.com' # sign in, New document; note ?doc=<id>
```

1 · TOKENS AND THE THEME (E10.1)

```
# press Dark theme in the bar: every surface flips; the page frame stays a white sheet on a dark desk
# reload: still dark (localStorage noon.theme); in DevTools > Rendering, emulate prefers-color-scheme: light: still dark (the choice outranks the OS)
# a fresh private window follows the OS live
pnpm exec vitest run apps/web/src/theme.test.ts # every text/surface pair AA in both themes, recomputed from tokens.css
```

2 · THE CANVAS (E10.2)

```
# ctrl/cmd+wheel over the Button: the point under the pointer stays put; the readout bottom-right changes; the outline's stroke does not thicken
# + and - zoom about the centre; 0 and Fit refit; Space+drag and middle-drag pan; the clamp is 10 %..400 %
# click a Button inside a Card: the deepest box wins; arrows walk siblings, Enter goes in, Shift+Enter out, Escape to the page
# try to click the Button as a button: nothing happens (inert); Tab from the canvas leaves it in one stop
```

3 · ONE MOVE_NODE PER DRAG (E10.3)

```
# DevTools > Network > WS > the /documents/ socket > Messages. Drag Text 1 onto the middle of Card 1 (a box), let go:
# exactly ONE frame {"type":"op",...,"op":{"type":"move_node","nodeId":...,"newParentId":...,"index":N}} goes up
# drag it to the top quarter of Button 1 (a 2 px line): one frame, index 0. Alt+Down, Alt+Up, Alt+Left: one frame each
# Alt+Right onto a Button: the line, then "Button does not take children" in the notice strip; no frame was refused by the
```

```
tree, one was by the replica
# in the other window: every move appears live in its tree and on its canvas
```

4 · THE INSPECTOR (E10.4)

```
# select a Stack: Layout shows direction and align as arrow segments, gap as a stepper; hover gap: the spaces between the
children shade on the canvas
# type 24 into gap and press Tab: ONE set_prop frame (watch the socket); type 1,2,3,4 slowly: still one frame, on blur
# Up/Down step by 1 and send at once; Shift+Up by 10; Reset appears once set and sends set_prop null
# in the other window as a viewer (?user=viewer@example.com, made a viewer on the org page): edit a prop: the refusal repeats
under the control
```

5 · THE LIBRARY (E10.5)

```
# drag the Card tile over the layers: a box round a container, a line beside a leaf, exactly as a dragged layer shows
# drag the Button tile onto the canvas over the Card: a box round the Card and a line among its children; let go: ONE add_node
frame; the new node is selected
# select a Text, press Enter on a tile: added AFTER the text (a leaf), and selected
```

6 · CURSORS IN WORLD COORDINATES, AND THE AI'S (E10.6)

```
open 'http://localhost:5199/?doc=$DOC&user=bob@example.com' # a second window; zoom it to 32 % with -, and the first to 160
% with +
# rest Bob's pointer on the Button: in Ada's window the arrow's tip is on HER Button, at her zoom; zoom Ada's window: it stays
on the Button
# hold still 5 s: it fades; move: it is back. Bob selects the Card: Ada presses Bob's avatar in the bar: her selection is the
Card, centred
# Ask the AI for 'a card with two buttons': a spark tagged AI sits on the node its last accepted op touched, moving as the run
goes
```

7 · THE PREVIEW SPLIT AND THE SHEET (E10.7)

```
# press P (or Preview in the bar): the running page opens beside the canvas in a phone frame at 390 CSS px, scaled to fit;
Tablet and Desktop resize the frame, not the page's idea of its width
# drag the divider, or focus it and press Left/Right; press P again: the preview unmounts (the container is released after a
minute)
# press ?: the sheet lists every key by scope; Tab cycles inside it (focus is trapped); Escape closes it and focus returns to
where it was
# type ? into the library's search box: it is a ? (typingIn)
```

8 · MEMBERS AND SHARE (E10.8)

```
open 'http://localhost:5199/?org=$ORG&user=ada@example.com' # the members with roles; as an owner, selects and an Add form
# add bob@example.com as viewer; change him to editor; try to make yourself, the only owner, an editor: "An organisation must
keep at least one owner..."
open 'http://localhost:5199/?org=$ORG&user=bob@example.com' # the same list, no controls: the api decides, the screen only
hides
# in the document as Ada: Share in the bar (Bob's bar has none); share with carol@example.com (signed up) as viewer; Carol's
window opens it live
# Revoke: the confirmation appears in place with focus on Revoke; Keep returns focus to the button; Revoke closes Carol's
session (her status says closed)
```

9 · THEN THE CHECKS THAT PROVE IT WITHOUT HANDS

```
pnpm exec vitest run apps/web/src # every pure half of the editor
pnpm exec vitest run --config vitest.integration.config.ts apps/api/src/members-shares.int.test.ts
pnpm exec playwright test e2e/shell-light-dark.spec.ts e2e/canvas-zoom-pan-select.spec.ts e2e/layers-drag-and-keyboard-
reorder.spec.ts e2e/inspector-edits-propagate.spec.ts \
  e2e/library-drag-to-canvas.spec.ts e2e/cursors-agree-across-zoom.spec.ts e2e/ai-cursor-on-touched-node.spec.ts e2e/preview-
split-device-frames.spec.ts \
  e2e/shortcut-sheet.spec.ts e2e/members-role-change.spec.ts e2e/share-dialog.spec.ts
pnpm exec playwright test --update-snapshots e2e/shell-light-dark.spec.ts e2e/home.spec.ts # the baselines changed with the
tokens; look at the diff before accepting
make check # the whole gate, as the pre-commit hook runs it
```

What to do when a step does not match this page: the page was cut from the code at commit `f21d6d2`, and the code is right until a test says otherwise. A step that fails is a finding: write it as *sequence* → *false belief* → *consequence* → *smallest fix* (SPEC §4a's form) and file it against the bead that owns the code.

13 · What the type system could not see, epic 10 edition

Everything below compiled under `strict`, passed the linter and the first tests, or was rejected at design time by asking what the types could not ask.

WHAT WENT WRONG, OR WOULD HAVE	WHY	FIX
a peer opened twice under StrictMode	"create it once" in <code>useMemo</code>	open in <code>useEffect</code> with a cleanup
a snapshot that never changed	<code>peer.doc</code> is the same object	a string of the numbers that move
a stale selection after someone else's remove	state is state	derive the node from <code>wanted</code> each render
the canvas re-rendering sixty times a second while panning	a drag point in state	a ref
an effect that set state on every run	measuring is measuring	return the same object when nothing moved
a flash of the wrong theme	the effect applies it	<code>applyStoredTheme()</code> before the first render
a dark token left out showing light on dark	the cascade inherits	a test that the dark block names every colour
a colour at 3:1 on a dark surface	"it looked fine on white"	contrast recomputed for every pair, both themes
the editor's <code>input</code> rule painting the sample app's field dark	one stylesheet	the tokens redefined on the frame to <code>--ds-*</code>
the seed's <code>:root</code> restyling the editor	a stylesheet is a stylesheet	scoped to the frame's class on load
a border stroke thickening with the zoom	2px / zoom is 2px	a box-shadow spread; Chrome snaps borders
ctrl+wheel zooming the browser	<code>onWheel</code> with <code>preventDefault</code>	a native non-passive listener
the shallowest element winning the click	<code>elementFromPoint</code>	the deepest measured box
the Button on the canvas taking the click	it is a real Button	<code>inert</code>
a corrupt document with a cycle hanging the tab	a parent walk ends	a <code>seen</code> set
a move landing one too far	index = position of the target	counted after the node is taken out
a remove and an add for a move	"take it out, put it in"	one <code>move_node</code> (drill 2)
the tree predicting <code>validate()</code>	grey out the leaf	build the op; the replica refuses
a control inside a treeitem	a disclosure button	a span; Left/Right are its keys
Tab changing the selection	select on focus	arrows select; a bare focus does not
a prop rendering as nothing	an unknown kind falls through	<code>never</code> and a throw
<code>gap: string</code> shaded as a space	keyed on the name	name AND kind
one <code>set_prop</code> per keystroke	<code>onChange</code> sends	commit on blur or Enter
"12abc" deleting a prop	<code>value === ""</code>	<code>validity.badInput</code>
a radio group unnamed	a fieldset has a legend	<code>aria-labelledby</code> a span
a spin that changed the field and sent nothing	native spin arrows	buttons that commit
a refused add leaving a selection on a ghost	select the new id	select only if the replica took it; back to the page on refusal
a tile fetching an image called "Image"	<code>src</code> is a string	an inline SVG
a cursor as a fraction of the canvas	the E2.6 answer	a world point, bounded, to the hundredth
a cursor never fading	no event when a pointer rests	a 1 s tick reading the clock

a presence refresh counted as a move	the same spot arrived again	keep <code>movedAt</code> when unchanged
a render for every person's op in the AI's anchors	a new Map each time	the same Map back
a cursor that talks	<code>aria-live</code> on the mark	an <code>aria-hidden</code> layer; the bar is the record
a key the sheet promised that did nothing	two lists	one registry drives both
two rows on one key	the last one wins	a test that no two collide
<code>?</code> in the search box opening the sheet	global is global	<code>typingIn</code>
a hand-rolled focus trap	modals need one	<code><dialog>.showModal()</code>
the page seeing the scaled width	scale the iframe	a frame of the real width, scaled by transform
a bordered 390 leaving the page 374	a border is a bezel	a box-shadow
a 403 in every viewer's console	probe <code>/shares</code>	<code>role</code> beside the token
a role in the token	"the token knows"	the row, read per request and per op
a guessed member row	optimistic everywhere	the api's answer only; a 403 re-reads
focus falling out of the dialog on revoke	the row is gone	focus to the email field
a name rendered as markup	a string is a string	React text nodes
six copies of the e2e scaffolding	each spec stands alone	<code>e2e/editor.ts</code> ; jscpd said so
an unused export hidden by a knip ignore	"it is fine"	make it a plain function

The pattern, extended once more: lesson 9 said types cannot see how many times a call happens or which clock a number came from. Epic 10 adds the two that make a UI hard: types cannot see *which coordinate system* a number is in (a screen px, a world px and a fraction are all `number`), and cannot see *which copy is the truth* (a widget's field and the replica's node have the same type). Both were answered as before: put the truth in one place (the replica, the registry, the manifest, `tokens.css`) and derive everything else from it on every render; name the coordinate system in the function's name (`toWorld`, `toScreen`) and convert at the edge, once; and write the test that provokes the exact case: two viewports and one point, a routed socket counting frames, the contrast of every pair, a manifest walked prop by prop.

Your turn: four drills

Do these alone. They live in `drills/lesson-10/` and the three coded drills start RED; `sh drills/lesson-10/check.sh` shows your progress. None needs Docker or a browser.

D1 · Extend the registry, end to end. The layers tree removes a node with Delete or Backspace; the canvas does not, so a person who selected a Button by clicking it on the sheet has to reach for the tree. Give the canvas the key the way every key in the editor is given: ONE row in `apps/web/src/shortcuts.ts` (`scope: "canvas", action: "remove"`), which makes `ActionOf<"canvas">` know `"remove"` and puts the row on the `?` sheet with no change to `ShortcutSheet.tsx`; `Surface.tsx` gains an `onRemove: (id: string) => void` prop and a `case "remove":` that calls it for the selected node and never for the page; `Canvas.tsx` passes `onRemove={id => { submit({ type: "remove_node", nodeId: id }); }}`, the same `submit` every gesture uses. make check must stay green.

Predict first: the selected node was removed by someone else a moment before you press Delete. What does the replica answer at once, what does the room answer later, and what does the person see? (Section 1's "DERIVED, not stored" and `usePeer.ts`'s `quiet` hold the answer.)

D2 · Fix a planted bug. `drills/lesson-10/d2-one-move/moves.ts` is `dropToMoveOp` cut down to its decision with one bug from this chapter planted in it. Read the failing assertion and say, before you open the code, how many ops one drop sent, what

happened to the node's id, and what everyone else's selection of that node, the AI's anchor on it and the audit trail saw in between. The fix turns two ops into one, and counts the index after the node is taken out.

D3 · Fix a planted bug. `drills/lesson-10/d3-world-cursor/cursor.ts` is the surface's pointer send and cursor draw cut down to their decision with one bug planted. Read the failing assertion and say in which coordinate system the cursor was sent and which one the other window expected; then say why "a fraction of the canvas", epic 2's answer, would fail the same test. The fix is one call.

D4 · Explain a decision before reading mine. Answer out loud or on paper, then open the answer.

▼ 1. The layers tree, the canvas drop and the library all refuse only what is "structurally impossible on the document they can see" (the page, a node onto itself, beside the page) and build every other op, including a drop into a Button, which the replica then refuses. Defend that against the obvious alternative: disable the drop the moment the pointer is over a leaf, so the person never sees a refusal.

The rules that say what a document may become live in one place, `validate()` in the doc-model, and they are run twice on every op already: by the replica the moment `submit` is called (the synchronous verdict, so a refused drop is a notice within the same frame and the tree never shows the node moved) and by the room when it orders the op (the asynchronous verdict, for what the replica could not know: a parent someone removed meanwhile, a viewer whose role changed). A tree that greyed out leaves would be a third copy of the rule "this component takes no children", written in a different language (a CSS state instead of a refusal reason), and copies drift: the manifest gains a component that takes children conditionally, or the rule gains "not into its own ancestor at sequencing time", and the tree is wrong in a way no test of `validate()` can see. So the tree knows only geometry: the page has no parent (nowhere "beside" it), a node cannot be its own ancestor (a cycle), a node that is gone has no place. Those are facts about the tree's own rows, not about the document's rules, and they are the only refusals that would otherwise build an op with no meaning. The cost is one honest sentence ("Button does not take children") where a greyed row would have been silent; the gain is that the refusal path is the same for a person's drop, a slow room, a viewer and the AI, and the e2e spec asserts the sentence appears for Alt+Right into a Button rather than asserting a colour.

▼ 2. Cursors changed from "a fraction of the canvas" (epic 2) to world coordinates, and the AI's cursor is derived from the ops the room ordered through a new `onOp` hook on the peer rather than from presence. Why each, and why is `onOp` "a window onto who did what, not a write path"?

A fraction of the canvas element was right when every window drew the whole page in the same box at 100 %: half-way across meant the same component everywhere. With zoom and pan per window (E10.2) the canvas element is a viewport onto an infinite sheet, and half-way across one window is a different spot on the sheet from half-way across another; only a point on the sheet itself (the frame's own px at 100 %, what `toWorld` returns) names the same component in both, and the receiving side needs no conversion at all: the mark sits inside `.world` and the same `transform` that places the frame places it. The contract bounds the value at $\pm 1e6$ so a wild number never rides the wire, and `Surface` rounds to hundredths because a sub-pixel is noise. The AI has no pointer, so presence cannot carry where it is; but the room stamps every ordered op with its actor, and the peer already applies those ops, so telling a listener after each apply costs one call and no wire change.

`cursors.ts` keeps one anchor per agent actor id and hands back the same map for anyone else's op, so React renders nothing for a person's edits. It is a window because nothing in it can change the document: the listener is told after the replica applied the op, receives a value it cannot answer, and the peer does not depend on anyone listening; the ONE write path (keystone 2) is still `submit`, and a reader of `peer.ts` can see from the option's type that `onOp` returns `void`. Rejected: mapping a fractional cursor through the frame's box (breaks the moment either side pans), a cursor-per-node attribute in presence (a wire change beyond what world coordinates need), an `aria-live` on the cursors (decoration must not talk; the bar's list is the record).

▼ 3. The document is optimistic (the replica shows an edit before the room answers) but the members list and the Share dialog are not (a row changes only when the api has answered), and the session carries a `role` the editor shows Share by but never decides by. Defend all three, and say what a role change under an open page does.

The document can be optimistic because its ops were designed for it: they are small, the replica runs the same `validate()` the room runs so a refusal is rare and local, the invariant "optimistic = confirmed + pending in order" is tested as a property, and a refused op is simply dropped from `pending` and the guess rebuilt. A membership change has none of that: its truth is a Postgres row behind `need("owner")`, an advisory lock for the last owner, a 404 for an email nobody signed up with, and side effects (a revoked share closes a live session on every sync node); a guessed row could show "editor" for a person the api

refused, and there is no journal to rebuild from. So `withMember` applies the api's own answer, never the request, and a 403 or 404 makes the page read the list again because "the world moved under it". The role in the session is read from the same row that proved access when the session was minted, and it is *beside* the token, never inside it, because E1.5 stands: the token says who, and the routes and the room read the row for themselves on every request and every op. Showing Share only to an owner saves a viewer from a 403 in their console (the e2e fixture fails on any console error) and saves a route the api would refuse; it decides nothing, because pressing Share on a stale page reaches `GET /shares` with `need("owner")`, and the api's refusal comes back as a sentence in the dialog. A role change under an open page is therefore met at the next action, as a sentence, and the room meets it sooner: it re-reads the role on every Redis hint and at the 30 s sweep (E8.3), so a demoted editor's next op is refused by the room whatever the screen still shows.

Recall first

Look away. Name the two documents the replica holds and the one number a React component subscribes to. Then say what a drag on the layers tree sends and how its index is counted. Then check.

▼ Check yourself

`confirmed` (what the server said, in its order) and `optimistic` (confirmed plus every pending op of ours, in order); the UI subscribes to `revision`, which goes up whenever the optimistic document changes, because the document is edited in place and its identity says nothing (`useSyncExternalStore`'s snapshot is a string of revision, pending count, status, read-only and presence revision). A drag sends exactly one `move_node {nodeId, newParentId, index}` on pointer-up, never a remove and an add and never one per pixel; the index is the node's FINAL place among its new siblings counted *after* it was taken out, which is why the siblings are filtered before `indexOf`.

Then test it

1. A WebSocket message arrives with another person's `set_prop`. How does the inspector on your screen learn about it?

The peer calls `setState` on the inspector

The message is dispatched to the DOM as a custom event the inspector listens for

The replica applies it and bumps `revision`; the store's listeners are told; `useSyncExternalStore` reads a new snapshot string, so React re-renders Canvas from `peer.doc`, and the inspector, a child, receives the new node as a prop

The inspector polls `peer.doc` every 100 ms

No component holds a copy of the document: every pane renders from the one replica, which is why another person's move shows in the tree, the canvas and the inspector at once.

2. Why is the dark theme `:root[data-theme="dark"]` and not `@media (prefers-color-scheme: dark)`?

The attribute is faster to match

Media queries cannot change custom properties

The attribute carries the RESOLVED theme, set before the first paint from the remembered choice or the OS; one dark block serves both cases, and a toggle that outranks the OS needs no second copy of every token under a media query

Playwright cannot emulate prefers-color-scheme

`useTheme` still reads the OS through `matchMedia`, as an external store, so a browser that never chose flips live when the OS does; the spec asserts exactly that.

3. You zoom to 300 %. What happens to the selection outline's 2 px stroke and to the outline's position?

The frame is re-rendered at the new size and the outline follows on the next frame

Every outline is re-measured with `getBoundingClientRect` on each wheel event

Nothing is re-measured: the outline lives in `.world` under the one `translate()` `scale()`, its box is stored in world coordinates, and its stroke is `calc(2px / var(--zoom))`, so it stays 2 px on the screen

The stroke becomes 6 px, as Figma's does

The measuring pass runs after a render, not after a zoom; a box-shadow spread was chosen over a border because Chrome snaps border widths to whole pixels.

4. A person drags "Text 2" and drops it after "Button 1", which sits inside a Card. What goes over the wire?

One `move_node {nodeId: text2, newParentId: card, index: 1}` on pointer-up; the index is counted among the card's children with `text2` filtered out first

A `remove_node` then an `add_node` with a fresh id

One `move_node` per pointer-move event while the line is shown

Nothing until the room confirms the drop is allowed

The e2e spec routes the WebSocket and counts `move_node` frames, so "one op per move" is measured, not assumed.

5. The manifest is regenerated and Card gains a `subtitle: string` prop. What changes in the inspector's code?

A new case "Card" in the inspector

Nothing: `controlFor` maps the prop's KIND (string → text field), `splitProps` puts it in Props by name and kind, and `controls.test` walks the committed manifest to prove every prop has a control

A new field component per prop name

The prop renders as nothing until someone adds a control

The one thing that would change code is a new prop *kind*: `controlFor`'s `default` branch types the value as `never` and throws, so a prop without a control can never render as nothing.

6. Ada is at 50 %, Bob at 240 %. Ada's pointer rests on the Button. What does Ada's browser send, and what does Bob's do with it?

Ada sends the node id under the pointer; Bob draws on that node

Ada sends a fraction of her canvas element; Bob multiplies by his canvas size

Ada sends her pointer's client px; Bob scales them by 2.4 / 0.5

Ada sends `toWorld(viewport, pointer)` rounded to hundredths, a point on the sheet; Bob draws the mark at that point inside `.world`, so his own transform lands it on his Button, and the mark divides by --zoom to keep its screen size

The node id is how the AI's cursor works, because the AI has no pointer; a person's pointer can rest on empty sheet, which is a place too.

7. Why do Surface, LayersPanel, LibraryPanel and Inspector call `actionFor(scope, event)` instead of comparing `event.key`?

React normalises `event.key` differently per browser

One table drives the handlers AND the ? sheet, so a key the sheet promises is a key that works; the action type per scope is derived from the table, and a test fails on two rows one keydown could mean

So Ctrl and Cmd chords can be bound later

Because `event.key` is deprecated

Ctrl and Cmd are never bound on purpose: `actionFor` returns null for them, so browser chords stay the browser's.

8. An owner has the org page open; another owner demotes them to editor. They change someone's role. What happens?

The change is applied on the screen and silently reverted later

The page crashes on the 403

The WebSocket pushes the new role and the select disappears first

The select is still shown (the screen's hint is stale), the api answers 403, the page shows the sentence for forbidden, drops its local changes and reads the list again; nothing was guessed onto the screen

The members list is deliberately not optimistic: `withMember` only ever applies the api's own answer.

Next: Lesson 11, after epic 11.